



Amazon Simple Storage Service (S3)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Introduction to S3

What is Object Storage (Object-based Storage)?

Object storage is a data storage architecture that **manages data as objects**, **as opposed** to other storage architectures.



- S3 provides you with ***unlimited storage**.
- You don't need to think about the underlying infrastructure
- The S3 Console provides an interface for you to upload and access your data



S3 Object

Objects contain your data. They are like files.

Object may consist of:

- **Key** this is the name of the object
- **Value** the data itself made up of a sequence of bytes
- **Version ID** when versioning enabled, the version of object
- **Metadata** additional information attached to the object



S3 Bucket

Buckets hold objects. Buckets can also have folders which in turn hold objects

S3 is a universal namespace, so bucket names must be unique (think like having a domain name)

You can store an individual object from **0 Bytes** to **5 Terabytes** in size

S3 Bucket Overview



S3 Buckets are infrastructure, and they hold S3 Objects

- **S3 Bucket Naming Rules** – how we have to our name bucket
- **S3 Bucket Restrictions and Limitations** – What we can and can't do with buckets
- **S3 Bucket Types** – the two different kinds of buckets, flat (general purpose) and directory
- **S3 Bucket Folders** – How S3 Buckets has virtual folders for general purpose buckets
- **Bucket Versioning** – how we can version of all objects
- **Bucket Encryption** – how we can encrypt the contents of bucket
- **Static Website Hosting** – How we can let our buckets host websites

S3 is a globally available service, but when you create a bucket, you **specify a region**.

S3 Bucket Naming Rules

S3 Buckets Names are similar to valid URL rules (but more rules). This is because S3 bucket names are **used to form URL links** to perform various HTTPs operations.


`https://myexamplebucket.s3.amazonaws.com/photo.jpg`

- **Length:** Bucket names must be **3-63 characters** long.
- **Characters:** Only lowercase letters, numbers, dots (.), and hyphens (-) are allowed.
- **Start and End:** Bucket names must **begin and end with a letter or number**.
- **Adjacent Periods:** No two adjacent periods are allowed.
- **IP Address Format:** Names cannot be formatted as IP addresses (e.g., 192.168.5.4).
- **Restricted Prefixes:** Can't start with "xn--", "s3alias-", "amzn-s3-demo", "sthree-", or "sthree-configurator".
- **Restricted Suffixes:** Can't end with "-s3alias", "--ol-s3", "--x-s3", ".mrp" reserved for access point alias names.
- **Uniqueness:** Must be unique across all AWS accounts in all AWS Regions within a partition.
- **Exclusivity:** A bucket name cannot be reused in the same partition until the original bucket is deleted.
- **Partitions:** "aws" (Standard Regions), "aws-cn" (China Regions), "aws-us-gov" (AWS GovCloud US)
- **Transfer Acceleration:** Buckets used with S3 Transfer Acceleration can't have dots in their names.



No Uppercase, No Underscores, No Spaces in Bucket Names

S3 Bucket Naming Rules – Examples

	 Valid	1. mybucket-123
<i>formatted as an IP address</i>	 Invalid	2. 123.456.789.012
<i>contains uppercase letters</i>	 Invalid	3. My-Bucket
<i>contains adjacent periods</i>	 Invalid	4. data.bucket..archive
	 Valid	5. log-bucket
<i>starts with "xn--"</i>	 Invalid	6. xn--bucketname
<i>contains an underscore</i>	 Invalid	7. bucket_name
	 Valid	8. example-bucket-1
<i>starts with "sthree-config"</i>	 Invalid	9. sthree-config-bucket
	 Valid	10. test.bucket.data
<i>ends with "-s3alias"</i>	 Invalid	11. new-bucket-s3alias
	 Valid	12. archive-bucket

S3 – Bucket Restrictions and Limitations

- **Default bucket limit per AWS account:** **10,000 general-purpose buckets**. You can request an increase via **Service Quotas** if needed
- **No cost or billing tied to number of buckets.** AWS bills based on actual usage (e.g., storage, operations), not bucket quantity

Bucket & Object Constraints

- **Objects per bucket:** Unlimited number of objects allowed
- **Object size:** Range is **0 bytes up to 5 TB**. Usage of **multipart upload** is mandatory for files larger than 5 GB

Important Operational Guidelines

- **Bucket creation/deletion:** Must be **empty before deletion**. Use these operations judiciously. They are not designed for frequent runtime use but rather for setup/init routines
- **ListBucket API:** If your bucket quota exceeds 10,000, **only paginated** `ListBuckets` **requests are supported**. Unpaginated versions will be rejected

S3 – Bucket Restrictions and Limitations

Access Points & Variants (Advanced Use Cases)

- **S3 Access Points:** Enable fine-grained access control to buckets. Default limit: **10,000 access points per Region per account**, with no hard cap; can request increases
- **Bucket Types** (new, beyond general-purpose):
 - **Directory buckets:** Used for low-latency, single-AZ cases. Default limit is **100 per account**
 - **Table buckets:** Designed for tabular data (Apache Iceberg) storage. You can create **up to 10 per Region per account**, and each supports up to **10,000 tables**
 - **Vector buckets:** Optimized storage for ML embeddings. No limits cited yet, but they are becoming increasingly relevant

S3 – Bucket Types

Amazon S3 has two types of buckets:



General purpose buckets

- Organizes data in a **flat hierarchy**
- The original S3 bucket type
- Recommended for most use cases
- Used with all storage classes except can't be used with S3 Express One Zone storage class
- There aren't prefix limits
- There is a default limit of 100 general buckets per account



Directory buckets

- Organizes data **folder hierarchy**
- Only to be used with **S3 Express One Zone storage class**
- Recommended when you need single-digit millisecond performance on PUT and GET
- *There aren't prefix limits for directory buckets
- Individual directories can scale horizontally
- There is a default limit of 10 directory buckets per account

S3 – Bucket Folder



The S3 Console allows you to “create folders”. **S3 general purpose buckets** do not have true folders found in hierarchy file system.

When you create a folder in S3 Console, Amazon S3 creates a **zero-byte** S3 object with a name that ends in a forward slash eg. **myfolder/**

☐	Name ▾	Type ▲	Size ▾
☐	 myfolder/	Folder	-
☐	 myfile.jpg	jpg	41.5 KB



- S3 folders are not their own independent identities but just S3 Objects
- S3 folders don't include metadata, permissions
- S3 folders don't contain anything, they can't be full or empty
- S3 folders aren't “moved”, S3 objects containing the same prefix are renamed

S3 – Bucket Folder

```
aws s3api list-objects --bucket testbucket-ab123
```

Folders will be **represented as an object**

Folders will be **0 bytes** in size.

An object in a folder will have a **prefix** to represent the folder

```
{
  "Contents": [
    {
      "Key": "myfolder/",
      "LastModified": "2023-12-05T17:12:51+00:00",
      "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
      "Size": 0,
      "StorageClass": "STANDARD",
      "Owner": {
        "DisplayName": "developers",
        "ID": "74ababf54b6810c1d34431."
      }
    },
    {
      "Key": "myfolder/myfile.jpg",
      "LastModified": "2023-12-05T17:15:18+00:00",
      "ETag": "\"39ddb20c1b578ca97d1673bceaa7ac9c\"",
      "Size": 42509,
      "StorageClass": "STANDARD",
      "Owner": {
        "DisplayName": "developers",
        "ID": "74ababf54b6810c32fg..."
      }
    }
  ]
}
```

S3 Object Overview

 **S3 Objects** are resources that **represent data** and is not infrastructure.

- **Etags** — a way to detect when the contents of an object has changed without download the contents
- **Checksums** — ensures the integrity of a files being uploaded or downloaded
- **Object Prefixes** — simulates file-system folders in a flat hierarchy
- **Object Metadata** — attach data alongside the content, to describe the contents of the data
- **Object tags** — benefits resource tagging but at the object level
- **Object Locking** — makes data files immutable
- **Object Versioning** — have multiple versions of a data file

S3 Objects – ETags

What is an ETag?

An entity tag (etag) is a response header that **represent a resource that has changed (without the need to download).**

- The value of an etag is generally represented by a hashing function eg. MD5 or SHA-1
- Etags are part of the HTTP protocol.
- Etags are used for revalidation for caching systems.

S3 Objects **have an etag**

- etag represents a hash of the object
- reflects changes only to the contents of an object, not its metadata
- may or may not be an MD5 digest of the object data (depends if it's encrypted)
- Etag represents a specific version of an object

```
{
  "Contents": [
    {
      "Key": "example-object-1.txt",
      "LastModified": "2023-11-22T10:15:00+00:00",
      "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
      "Size": 0,
      "StorageClass": "STANDARD"
    },
    ...
  ]
}
```



ETags are useful if you want to programmatically detect content changes to S3 objects

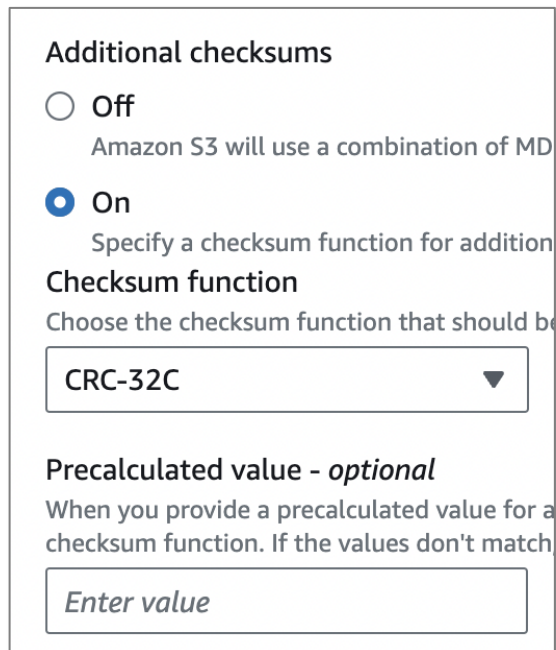
aws s3api list-objects

S3 Object – Checksums

What is a Checksum?

A checksum is used to **check the sum (amount) of data to ensure the data integrity of a file.** If data is downloaded and if in-transit data is loss or mangled the checksum will **determine this is something wrong with the file.**

Amazon S3 uses checksums to verify data integrity of files on the upload or download.



Additional checksums

☐ Off
Amazon S3 will use a combination of MD5 and CRC32 checksums.

☒ On
Specify a checksum function for additional verification.

Checksum function
Choose the checksum function that should be used for additional verification.

CRC-32C ▼

Precalculated value - *optional*
When you provide a precalculated value for a checksum function, the checksums must match. If the values don't match, the object is rejected.

Enter value

AWS allows you to **change the checksum algorithm** during uploading an object

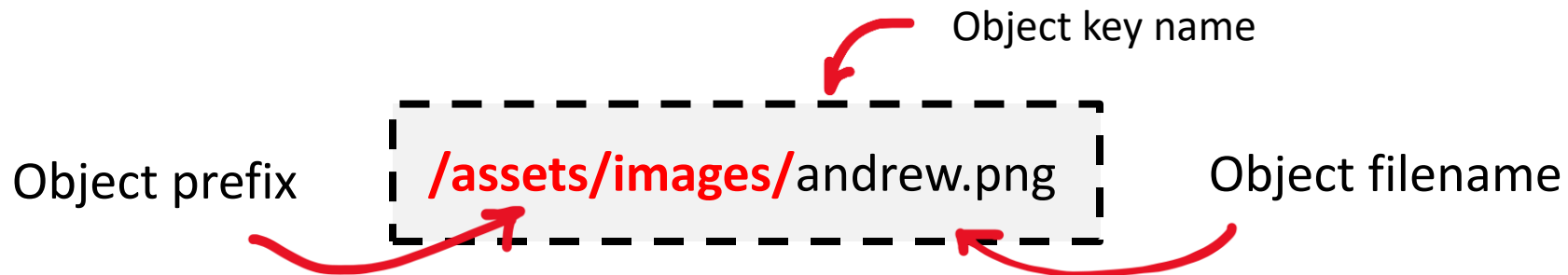


Amazon S3 offers the following checksum algorithms

- CRC32 (Cyclic Redundancy Check)
- CRC32C
- SHA1 (Secure Hash Algorithms)
- SHA256

S3 Object – Prefixes

S3 Object prefixes are **strings that proceed** the **Object filename** and is part of the Object key name



Since all objects in a bucket are stored in a **flat-structured hierarchy**, Object prefixes allows for a way to **organize, group and filter** objects.

A prefix uses the forward slash `/` as a delimitator to group similar data, similar to directories (folders) or subdirectories. Prefixes are **not true folders**.



There is no limit for the number of delimiters, the only limit the Object key name cannot exceed 1024 bytes with the object prefix and filename combined.

S3 Object – Metadata

What is Metadata?

Metadata provides information about other data but not the contents itself.

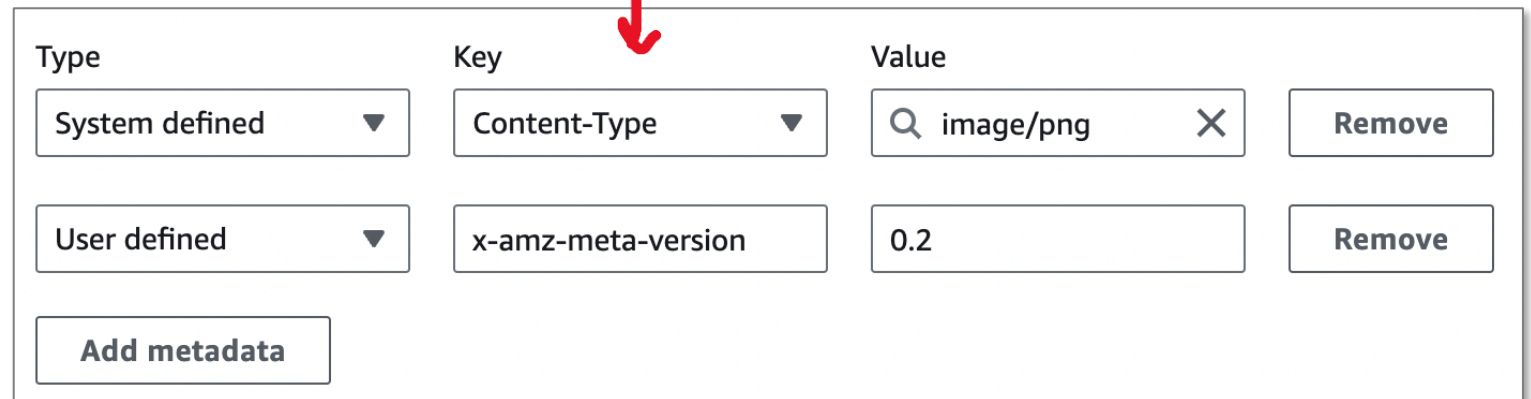
Metadata is useful for:

- categorizing and organizing data
- providing content about data

Amazon S3 allows you to **attach metadata to S3 Objects** at anytime

Metadata can be either:

- System defined
- User-defined



Type	Key	Value	
System defined ▼	Content-Type ▼	image/png	Remove
User defined ▼	x-amz-meta-version	0.2	Remove

Add metadata

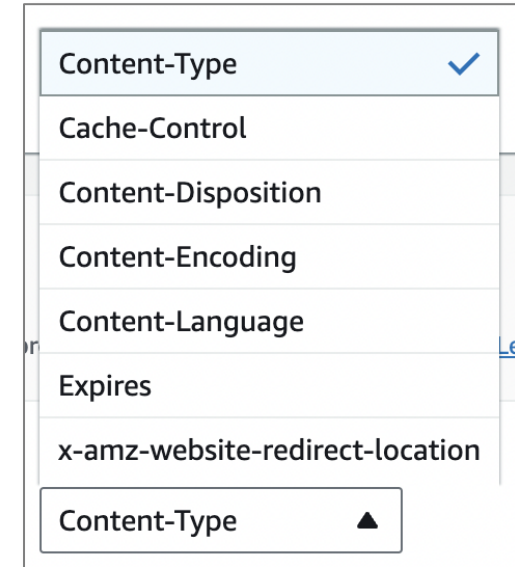


Resource Tags and Object tags are similar to Metadata, but tags are intended to provide information **about cloud resources** (eg. S3 Objects) and not the contents of the object.

S3 Object – System Defined Metadata

System Defined Metadata is data that only Amazon can control.
User's usually* cannot set their values for these metadata values.

- **Content Type:** image/jpeg
- **Cache Control:** max-age=3600, must-revalidate
- **Content Disposition:** attachment; filename='example.pdf'
- **Content-Encoding:** gzip
- **Content-Language:** en-US
- **Expires:** Thu, 01 Dec 2030 16:00:00 GMT
- **X-amz-website-redirect-location:** /new-page.html



A screenshot of the S3 metadata settings interface. It shows a list of system-defined metadata fields: Content-Type (selected with a blue checkmark), Cache-Control, Content-Disposition, Content-Encoding, Content-Language, Expires, and x-amz-website-redirect-location. Below the list is a 'Content-Type' field with an upward arrow icon.

There is more metadata than listed eg. etags

*Some system defined metadata can be modified by the user eg. Content Type

Metadata (2)			Edit
Metadata is optional information provided as a name-value (key-value) pair. Learn more			
Type	Key	Value	
System defined	Cache-Control	no-cache	
System defined	Content-Type	image/png	

AWS will attach some System defined metadata even if you do not specify any.

S3 –User Defined Metadata

User Defined Metadata is set by the user and must start with **x-amz-meta-** eg.

Access and Security:

x-amz-meta-encryption: "AES-256"
x-amz-meta-access-level: "confidential"
x-amz-meta-expiration-date: "2024-01-01"

Media File:

x-amz-meta-camera-model: "Canon EOS 5D"
x-amz-meta-photo-taken-on: "2023-06-10"
x-amz-meta-location: "New York City"

Custom Application:

-app-version: "2.4.1"
-data-imported: "2023-03-15"
-source: "CRM System"

Project-specific:

x-amz-meta-project-id: "PRJ12345"
x-amz-meta-department: "Marketing"
x-amz-meta-reviewed-by: "Jane Smith"

Document Versioning:

x-amz-meta-version: "v3.2"
x-amz-meta-last-modified-by: "Alice Johnson"
x-amz-meta-original-upload: "2023-02-01"

Content-related:

-title: "Annual Sales Report 2023"
-author: "John Doe"
-description: "Detailed sales ..."

Compliance and Legal:

x-amz-meta-legal-hold: "true"
x-amz-meta-compliance-category: "GDPR"
x-amz-meta-retention-period: "5 years"

Backup and Archival:

x-amz-meta-backup-status: "Completed"
x-amz-meta-archive-date: "2023-04-20"
x-amz-meta-recovery-point: "2023-04-15"

What is WORM?

What is WORM?

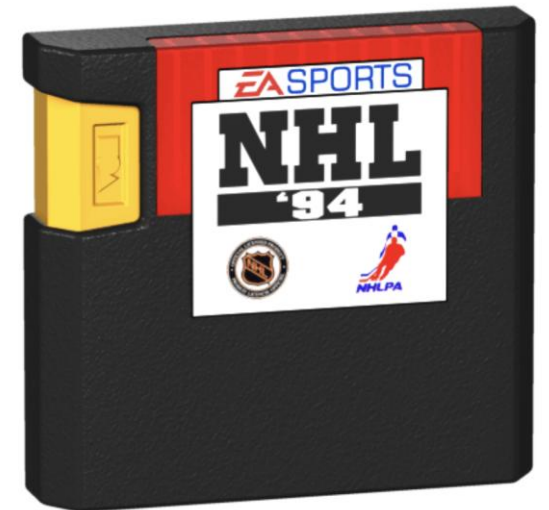
Write once read many (WORM) is a storage compliance feature that makes data immutable, You write once and the file can never be modified or deleted, but you may read the unlimited times



WORM is useful in healthcare or financial industries where files need to be audited and untampered

An example of WORM are video game cartridges.
Where you would write data permanently to a **ROM**
(Read only memory)

You can play the game (read the data) as many times as you want but you can't change the data.



S3 Object Lock



S3 Object Locks allows to **prevent the deletion of objects** in a bucket

This feature can only be turned on **at the creation** of a bucket

Object Lock is for companies that need to prevent objects being deleted to have:

- Data integrity
- Regulatory compliance

Object Lock

Store objects using a write-once-read-many model for a specified amount of time or indefinitely. [Learn more](#)

☐ Disable

☒ Enable

Permanently allows objects in this bucket to be deleted or overwritten after the retention period required in bucket details after bucket creation or overwriting.

S3 Object Lock is **SEC 17a-4**, **CTCC**, and **FINRA** regulation compliant

You can store objects using a **write-once-read-many (WORM) model** just like S3 Glacier.

You can use it to prevent an object from being deleted or overwritten for:

- a fixed amount of time
- or indefinitely.

Object retention is handled two different ways:

1. **Retention periods** — fixed period of time during which an object remains locked.
2. **Legal holds** — remains locked until you remove the hold

S3 buckets with Object Lock can't be used as destination buckets for server access log

S3 Object Lock – Setting Object Lock on Objects

Object Locking can be configured via AWS API (CLI, SDK) and the AWS Console

AWS API



```
aws s3api put-object \  
--bucket your-bucket-name \  
--key your-object-key \  
--body file-to-upload \  
--object-lock-mode GOVERNANCE \  
--object-lock-retain-until-date "2025-01-01T00:00:00Z"
```

This allows both technical and non-technical users to prevent object deletion or modification, ensuring compliance and data integrity

S3 Object Lock – Retention Holds

There are two modes for Retention modes: **Governance** and **Compliance**

Compliance Mode

- Protected object cannot be overwritten or deleted by any user, including root user.
- lock period cannot change or shorten.
- Ensures object version integrity throughout the retention period.
- Deleting an object in compliance mode requires deleting the associated AWS account.

Governance Mode

- Protected can't modify or delete without special permissions.
- Lock period can be changed or shorten or removed by special users
- Can be used for testing retention settings before enabling compliance mode.

The Special Permissions needed to override lock settings in Governance Mode

- **The IAM Permission:** s3:BypassGovernanceRetention
- **The request Header:** x-amz-bypass-governance-retention:true
- Use the aws **s3api put-object-retention** CLI Command to change the lock settings

S3 Object Lock – Legal Holds

Like a retention period, a legal hold prevents an object version from being overwritten or deleted
legal hold **doesn't have an associated fixed amount of time and remains in effect until removed.**

A user with the IAM Permission

- s3:PutObjectLegalHold

```
aws s3api put-object-legal-hold  
--bucket your-bucket-name \  
--key your-object-key \  
--legal-hold Status=ON
```



Just set **--legal-hold** flag to ON

- Legal holds are independent from retention periods
- Placing a legal hold on an object version doesn't affect the retention mode or retention period for that object version

Amazon S3 Bucket URI

The **S3 Bucket URI (Uniform Resource Identifier)** is a way to reference the address of S3 bucket and s3 objects.



s3://myexamplebucket/photo.jpg

You'll see this S3 Bucket URI required to be used for specific AWS CLI commands

```
aws s3 cp test.txt s3://mybucket/test2.txt
```



AWS S3 CLI

aws s3

A high-level way to interact with S3 buckets and objects

```
aws s3 cp test.txt s3://mybucket/test2.txt
```

aws s3api

A low-level way to interact with S3 buckets and objects

```
aws s3api put-object \  
--bucket text-content \  
--key dir-1/my_images.tar.bz2 \  
--body my_images.tar.bz2
```

aws s3control

Managing s3 access points, S3 outposts buckets, S3 batch operations, storage lens.

```
aws s3control describe-job \  
--account-id 123456789012 \  
--job-id 93735294-df46-44d5-8638-6356f335324e
```

aws s3outposts

Manage endpoints for S3 outposts

```
aws s3outposts create-endpoint \  
--outpost-id "op-0123456789abcdef0" \  
--subnet-id "subnet-0abcdef1234567890" \  
--security-group-id "sg-0abcdef1234567890" \  
--access-type "Private" \  
--customer-owned-ipv4-pool "ipv4pool-coip-abcdef1234567890"
```

S3 – Request Styles

When making requests by using the REST API there are two styles of request:

1. Virtual hosted-style requests — the bucket name is a subdomain on the host
2. Path-style requests — the bucket name is in the request path

Virtual hosted–Style request

```
DELETE /puppy.jpg HTTP/1.1
Host: examplebucket.s3.us-west-2.amazonaws.com
Date: Mon, 11 Apr 2016 12:00:00 GMT
x-amz-date: Mon, 11 Apr 2016 12:00:00 GMT
Authorization: authorization string
```

Path-style request

```
DELETE /examplebucket/puppy.jpg HTTP/1.1
Host: s3.us-west-2.amazonaws.com
Date: Mon, 11 Apr 2016 12:00:00 GMT
x-amz-date: Mon, 11 Apr 2016 12:00:00 GMT
Authorization: authorization string
```

S3 supports both virtual-hosted–style and path-style URL

Path-style URLs will be discontinued in the future

To force AWS CLI to use Virtual hosted-style requests, you need to **globally configure the CLI**



```
aws configure set s3.addressing_style virtual
```

S3 –Dualstack Endpoints

There are *two possible endpoints when accessing Amazon S3 API

Standard Endpoint

Handles only IPV4 traffic



`https://s3.us-east-2.amazonaws.com`

DualStack Endpoint

Handles both IPV4 and IPV6 traffic



`https://s3.dualstack.us-east-2.amazonaws.com`

At one point, AWS only offered IPV4 and Dualstack is designed as its future replacement since IPV4 addresses are running out and IPV6 has a larger public address space.

The AWS CLI likely uses dualstack underneath. Some AWS services may still generate out endpoints using the standard endpoint.

*There are other S3 endpoints for not being show here eg. Static Website, FIPS, S3 Controls, Access Points.

S3 Storage Classes Overview

AWS offers a range of S3 storage classes that *trade* **Retrieval Time, Accessibility and Durability** for **Cheaper Storage**



S3 Standard (default) Fast, Available and Durable.

S3 Reduced Redundancy Storage (RRS) legacy storage class



S3 Intelligent Tiering Uses ML to analyze object usage and determine storage class. Extra fee to analyze



S3 Express One-Zone single-digit ms performance, special bucket type, one AZ, 50% less than Standard cost



S3 Standard-IA (Infrequent Access) Fast, Cheaper if you access less than once a month. Extra fee to retrieve. **50% less** than Standard (reduced availability)



S3 One-Zone-IA

Fast Objects only exist in one AZ. Cheaper than Standard IA by 20% less (Reduce durability) Data could get destroyed. Extra fee to retrieve.



S3 Glacier Instant Retrieval For long-term cold storage. Get data instantly

S3 Glacier Flexible Retrieval takes minutes to hours get data (Standard, Expedited, Bulk Retrieval)

S3 Glacier Deep Archive The lowest cost storage class. Data retrieval time is 12 hours.

***Cheaper**



S3 Outposts has its own storage class.

S3 – Pricing

Data Transfer In

Data Transfer Out

Storage Per GB

Retrieval Fees

S3 Storage Classes – Standard



S3 Storage Standard is the **default storage class** when you upload to S3. It's designed for **general purpose storage for frequently accessed data**

High Durability: 11 9's of durability (99.999999999%)

High Availability: 4 9's of availability (99.99%)

Data Redundancy: Data stored in 3 or more Availability Zones (AZs)

Retrieval Time: within milliseconds (low latency)

High Throughput: optimized for data that is frequently accessed and/or requires real-time access

Scalability: easily scales to storage size and number of requests

Use Cases: Ideal for a wide range of use cases like content distribution, big data analytics, and mobile and gaming applications, where frequent access is required.

Pricing:

- Storage per GB
- Per Requests
- No Retrieval fee
- No minimum storage duration charge

S3 Storage Classes – RRS

S3 Reduced Redundancy Storage (RRS) is a legacy storage class to **store noncritical**, reproducible data at lower levels of redundancy than Amazon S3's standard storage

RRS was introduced in 2010, and at the time, it was cheaper than Standard storage.

In 2018, S3 Standard infrastructure changed, and the cost of S3 Standard storage fell well below the cost of RRS.

RRS currently provides no cost-benefit to customers for the reduced redundancy and has no place in modern storage use cases



RRS is no longer cost-effective and is not recommended for use.
It may appear in the AWS Console as an option due to legacy customers

S3 Storage Classes – Standard-IA



S3 Standard-IA (Infrequent Access) storage class is designed **for data that is less frequently accessed** but requires rapid access when needed.

High Durability: 11 9's of durability like S3 Standard

High Availability: 3 9's of availability (99.9%) Slower availability compared to S3 Standard

Data Redundancy: Data stored in 3 or more Availability Zones (AZs)

Cost-Effective Storage: costs 50% less from Standard. As long as you don't access a file more than once a month!

Retrieval Time: within milliseconds (low latency)

High Throughput: optimized for rapid access, although the data is accessed less frequently compared to S3 Standard.

Scalability: easily scales to storage size and number of requests like S3 Standard

Use Cases:

Ideal for data that is accessed less frequently but requires quick access when needed, such as disaster recovery, backups, or long-term data stores where data is not frequently accessed.

Pricing:

- Storage per GB
- Per Requests
- **Has a Retrieval fee**
- **Has a minimum storage duration charge of 30 days**

S3 Storage Classes – Express One Zone



Amazon S3 Express One Zone delivers **consistent single-digit millisecond data access** for your most frequently accessed data and latency-sensitive applications

- the lowest latency cloud object storage class available
- data access speeds up to **10x faster** than S3 Standard
- request costs **50% lower** than S3 Standard
- data is stored in a user selected **single Availability Zone** (AZ)
- data is stored in a new bucket type: **an Amazon S3 Directory bucket**



S3 Directory bucket supports simple real-folder structure, you are only allowed 10 by default S3 Directory buckets per account



Express One Zone applies a flat per request charge for request sizes up to 512 KB.

S3 Storage Classes – One-Zone-IA



S3 One-Zone IA (Infrequent Access) storage class is designed **for data that is less frequently accessed** and has additional saving at reduced availability.

High Durability: 11 9's of durability like S3 Standard and S3 Standard IA

Lower Availability: 99.5%. Since it's in a single AZ, it has even lower availability than Standard IA

Cost-Effective Storage: costs 20% less than Standard IA

Data Redundancy: Data stored in one Availability Zones (AZs) there is a risk of data loss in case of AZ disaster.

Retrieval Time: within milliseconds (low latency)

Use Cases:

Ideal for secondary backup copies of on-premises data, or for storing data that can be recreated in the event of an AZ failure. It's also suitable for storing infrequently accessed data that is **non mission-critical**.

Pricing:

- Storage per GB
- Per Requests
- **Has a Retrieval fee**
- **Has a minimum storage duration charge of 30 days**

S3 Glacier Storage Classes vs S3 Glacier "Vault"

S3 Glacier "Vault"



"S3" Glacier is a stand-alone service from S3 **that uses vaults** over bucket to store data long term.

S3 Glacier is the original vault service.

- It has vault control policies
- Most interactions occur via the AWS CLI
- Enterprises are still using S3 Glacier Vault

S3 Glacier Deep Archive is part of S3 Glacier "Vault"

S3 Glacier Storage Classes

S3 Glacier Storage classes offer similar functionality to S3 Glacier but with greater convenience and flexibility, all within s3 Buckets

This is a new class with no attachment to S3 Glacier Vault



S3 Glacier Instant

These two are using S3 Glacier Vault underneath



S3 Glacier Flexible



S3 Glacier Deep Archive



S3 Storage Classes – Glacier Instant Retrieval



S3 Glacier Instant Retrieval is a storage class designed for **rarely accessed data** that still needs **immediate access** in performance-sensitive use cases.

High Durability: 11 9's of durability like S3 Standard

High Availability: 3 9's of availability (99.9%) like S3 Standard IA

Cost-Effective Storage: **68%** lower cost than Standard IA,

- For long-lived data that is accessed once per quarter

Retrieval Time: within milliseconds (low latency)

Use Cases: Rarely access data that needs immediate access eg. image hosting, online file-sharing applications, medical imaging and health records, news media assets, and satellite and aerial imaging.

Pricing:

- Storage per GB
- Per Requests
- **Has a Retrieval fee**
- **Has a minimum storage duration charge of 90 days**



Glacier Instant Retrieval is not a separate service and does not require a Vault

S3 Storage Classes – Glacier Flexible Retrieval



S3 Glacier Flexible Retrieval (formally S3 Glacier) combines S3 and Glacier into a single set of APIs. It's considerably faster than Glacier Vault-based storage.

There are 3 **retrieval tiers**:
(the faster and more expensive)

Expedited Tier 1-5 mins

For urgent requests. Limited to 250 MB archive size

Standard Tier 3-5 hours

No archive size limit. This is the **default** option

Bulk Tier 5-12 hours

No archive size limit, even petabytes worth of data.
Bulk retrieval is free of charge (no cost per GB).

You pay **per GB retrieved** and **number of requests**
This is a separate cost from just the cost of storage.

Archived objects will have an additional **40KBs** of data:

- 32KB for archive index and archive metadata information
- 8KB for the name of the object

You should store fewer and larger files instead of smaller files. 40KBs on thousands of files add up.



Glacier Flexible Retrieval is not a separate service and does not require a Vault

S3 Storage Classes – Glacier Deep Archive



S3 Glacier Deep Archive combines S3 and Glacier into a single set of APIs. It's more cost-effective than S3 Glacier Flexible but greater cost of retrieval.

There are two **retrieval tiers**:

Standard Tier restores within 12 hours
No archive size limit. This is the **default** option

There is no expedited tier for Glacier Deep Archive

Bulk Tier restores within 48 hours
No archive size limit, even petabytes worth of data.

Archived objects will have an additional **40KBs** of data:

- 32KB for index and metadata information
- 8KB for the name of the object



Glacier Deep Archive is not a separate service and does not require a Vault

S3 Storage Classes – Intelligent-Tiering



S3 Intelligent-Tiering storage class **automatically moves objects into different storage tiers** to optimize cost and performance. A small monthly fee applies for **object monitoring and automation**

S3 Intelligent-Tiering access has the following access tiers:

- Frequent Access tier (automatic)
 - Default tier, objects remain as long as they are being accessed
- Infrequent Access tier (automatic)
 - Objects move here after **30 days** of inactivity
- Archive Instant Access tier (automatic)
 - Objects move here after **90 days** of inactivity
- Archive Access tier (optional)
 - After **90 days**, access on demand
- Deep Archive Access tier (optional)
 - After **180 days**, access on demand



S3 Intelligent-Tiering has additional cost for analyzing objects over a 30-day period

```
aws s3api put-object \  
--bucket my-bucket \  
--key 'myfile' \  
--body 'path/to/local/file' \  
--storage-class INTELLIGENT_TIERING
```

	Storage class	Designed for
☐	Standard	Frequently accessed data (more than once a month) with milliseconds access
☒	Intelligent-Tiering	Data with changing or unknown access patterns

S3 Storage Classes Comparison

	Express One-Zone	Standard	Intelligent Tiering	Standard IA	One- Zone IA	Glacier Instant	Glacier Flexible	Glacier Deep Archive
Durability	11 9's	11 9's	11 9's	11 9's	11 9's	11 9's	11 9's	11 9's
Availability	99.95%	99.99%	99.9%	99.9%	99.5%	99.9%	N/A	N/A
Availability SLA	99.9%	99.99%	99%	99%	99%	99%?	N/A	N/A
AZs	1	>3	>3	>3	1	>3	>3	>3
Min cap. charge per obj	N/A	N/A	N/A	128kb	128kb		40kb	40kb
Min stor. duration charge	N/A	N/A	30 days	30 days	30 days	90 days	90 days	180 days
Retrieval fee	N/A	N/A	N/A	Per GB	Per GB	Per GB	Per GB	Per GB
First byte latency	single-digit ms	ms	ms	ms	ms	ms	mins to hrs	hours

Additional charge for request
files larger than 512KB



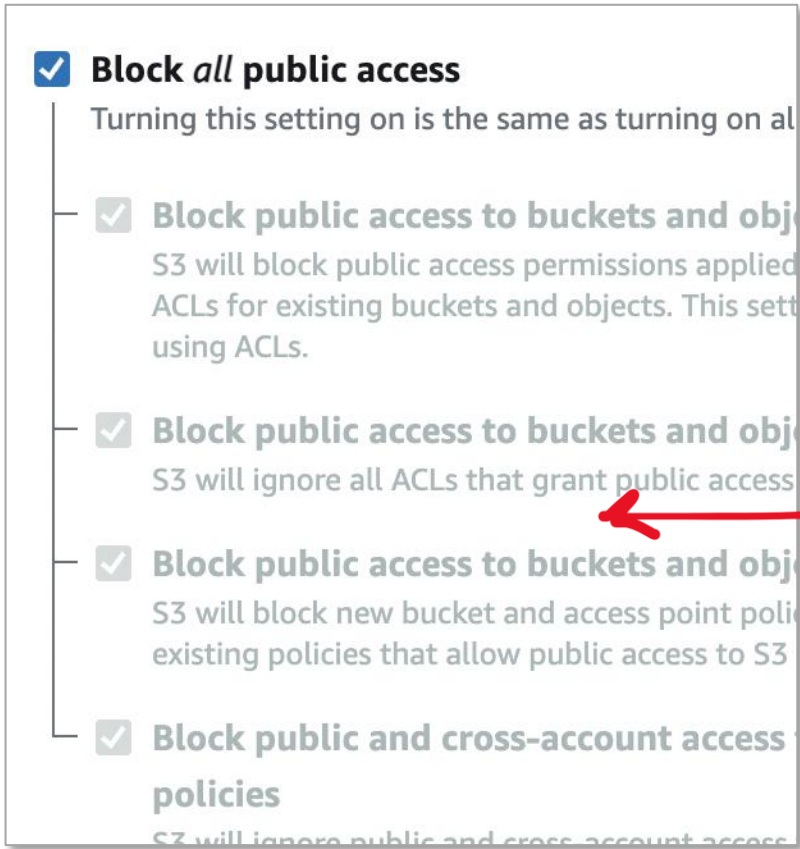
Additional monitoring cost

S3 Security Overview

- **Bucket Policies:** Define permissions for an entire S3 bucket using JSON-based access policy language.
- **Access Control Lists (ACLs):** Provide a legacy method to manage access permissions on individual objects and buckets.
- **AWS PrivateLink for Amazon S3:** Enables private network access to S3, bypassing the public internet for enhanced security.
- **Cross-Origin Resource Sharing (CORS):** Allows restricted resources on a web page from another domain to be requested.
- **Amazon S3 Block Public Access:** Offers settings to easily block public access to all your S3 resources.
- **IAM Access Analyzer for S3:** Analyzes resource policies to help identify and mitigate potential access risks.
- **Internetwork Traffic Privacy:** Ensures data privacy by encrypting data moving between AWS services and the Internet.
- **Object Ownership:** Manages data ownership between AWS accounts when objects are uploaded to S3 buckets.
- **Access Points:** Simplifies managing data access at scale for shared datasets in S3.
- **Access Grants:** Providing access to S3 data via a directory services e.g. Active Directory
- **Versioning:** Preserves, retrieves, and restores every version of every object stored in an S3 bucket.
- **MFA Delete:** Adds an additional layer of security by requiring MFA for the deletion of S3 objects.
- **Object Tags:** Provides a way to categorize storage by assigning key-value pairs to S3 objects.
- **In-Transit Encryption:** Protects data by encrypting it as it travels to and from S3 over the internet.
- **Server-Side Encryption:** Automatically encrypts data when writing it to S3 and decrypts it when downloading.
- **Client-Side Encryption:** Encrypts data client-side before uploading to S3 and decrypts it after downloading.
- **Compliance Validation for Amazon S3:** Ensures S3 services meet compliance requirements like HIPAA, GDPR, etc.
- **Infrastructure Security:** Protects the underlying infrastructure of the S3 service, ensuring data integrity and availability.

Amazon S3 Block Public Access

Block Public Access is a safety feature that is enabled by default to **block all public access to an S3 bucket.**



Unrestricted access to S3 buckets is the #1 security misconfiguration

There are four options:

- **New** Access Control Lists (ACLs)
- **Any** Access Control Lists
- **New** Bucket Policies or Access Points
- **Any** Bucket Policies or Access Points

Access points can have their own independent Block Public Access setting

Amazon S3 – Access Control Lists (ACLs)

ACLs grant basic read/write permissions to other AWS accounts

- you can grant permissions only to other AWS accounts
- you cannot grant permissions to users in your account
- You cannot grant conditional permissions
- You cannot explicitly deny permissions

S3 ACLs has been traditionally used to allow other AWS accounts to upload objects to a bucket



Access Controls Lists (ACLs) is a legacy feature of S3 and there are more robust ways to provide cross-account access via bucket policies and access points.

Amazon S3 – Bucket Policies

S3 Bucket Policy is a **resource-based policy** to grant an s3 bucket and bucket objects to other Principles eg. AWS Accounts, Users, AWS services

Only allow a specific role to read objects with prod object tag

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Principal": {
      "AWS": "arn:aws:iam::123456789012:role/ProdTeam"
    },
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:GetObjectVersion"
    ],
    "Resource": "arn:aws:s3:::my-bucket/*",
    "Condition": {
      "StringEquals": {
        "s3:ExistingObjectTag/environment": "prod"
      }
    }
  }]
}
```

Restrict access to specific IP

```
{
  "Version": "2012-10-17",
  "Id": "S3PolicyId1",
  "Statement": [{
    "Sid": "IPAllow",
    "Effect": "Deny",
    "Principal": "*",
    "Action": "s3:*",
    "Resource": [
      "arn:aws:s3:::my-bucket",
      "arn:aws:s3:::my-bucket/*"
    ],
    "Condition": {
      "NotIpAddress": {
        "aws:SourceIp": "192.0.2.0/24"
      }
    }
  }]
}
```

S3 Bucket Policies vs IAM Policies

S3 Bucket policies have overlapping functionality as an IAM policy that grants access to S3. S3 Bucket Policies provide convenience over IAM policies granting S3 access.

S3 Bucket Policy

Provides access to a specify bucket and its objects

You can specify multiple principals to grant access

Bucket Policies can be 20 KB in size

Block Public Access is turned on by default and will DENY all anonymous access even if the bucket policy grants it (unless the feature is turned off.)

IAM Policy

Provides access to many AWS services

Can provide permissions for multiple buckets in one policy

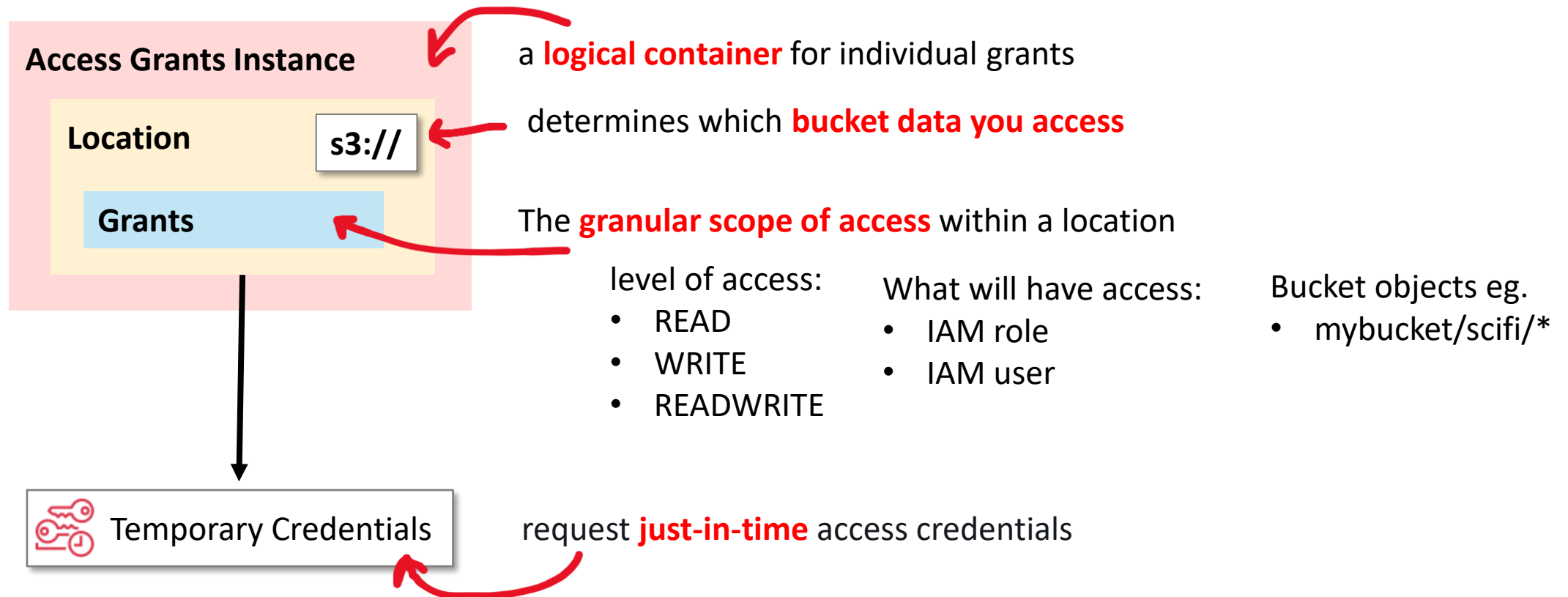
The principal, by default, is the entity the IAM policy is attached to

IAM policy sizes are limited based on the principal:

- Users 2 KB
- Groups 5 KB
- Roles 10 KB

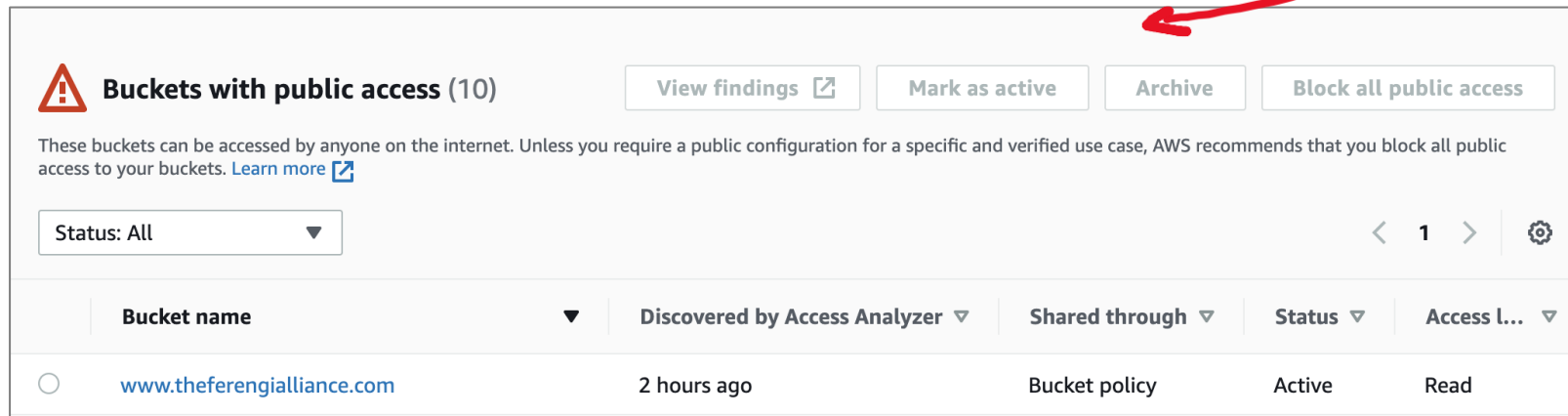
Amazon S3 – Access Grants

Amazon S3 Access Grants lets you **map identities in a directory service** (IAM Identify center, Active Directory, Okta) to access datasets in s3.



IAM Access Analyzer for S3

Access Analyzer for S3 will alert you when your S3 buckets are exposed to the **Internet** or **other AWS Accounts**



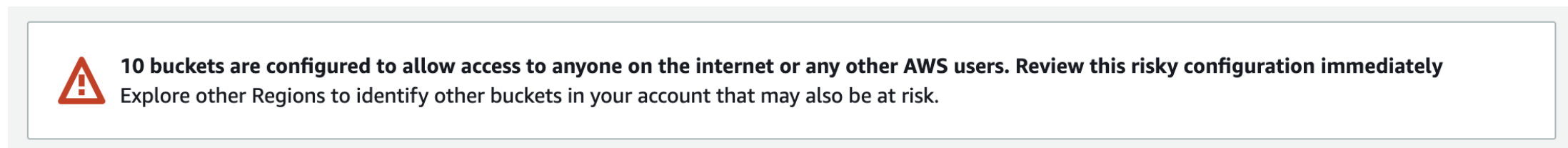
Buckets with public access (10) [View findings](#) [Mark as active](#) [Archive](#) [Block all public access](#)

These buckets can be accessed by anyone on the internet. Unless you require a public configuration for a specific and verified use case, AWS recommends that you block all public access to your buckets. [Learn more](#)

Status: All < 1 > [Settings](#)

	Bucket name	Discovered by Access Analyzer	Shared through	Status	Access l...
☐	www.theferengialliance.com	2 hours ago	Bucket policy	Active	Read

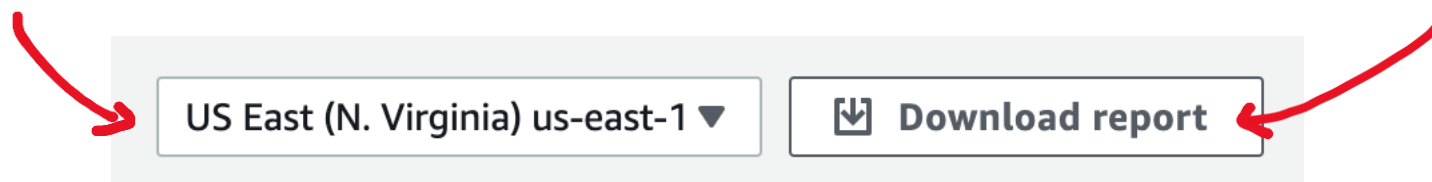
In order to use Access Analyzer for S3, you need to first create an analyzer in **IAM Access Analyzer** at the account level.



10 buckets are configured to allow access to anyone on the internet or any other AWS users. Review this risky configuration immediately
Explore other Regions to identify other buckets in your account that may also be at risk.

View buckets per region

Download a report



US East (N. Virginia) us-east-1 [Download report](#)

Amazon S3 – Internetwork traffic privacy

What is Internetwork traffic privacy?

Internetwork traffic privacy is about keeping data private as it travels across different networks.



AWS PrivateLink (VPC Interface Endpoints)

Allows you to connect an Elastic Network Interface Interface (ENI) directly to other AWS Services eg. S3, EC2, Lambda
It can connect to select Third-Party services via the AWS Marketplace.

AWS PrivateLink can go cross-account

Has fine-grain permissions via VPC endpoint policies

There is a charge for using AWS PrivateLink



VPC Gateway Endpoint

Allows you to connect a VPC directly to S3 (or DynamoDB), staying private within the internal AWS network.

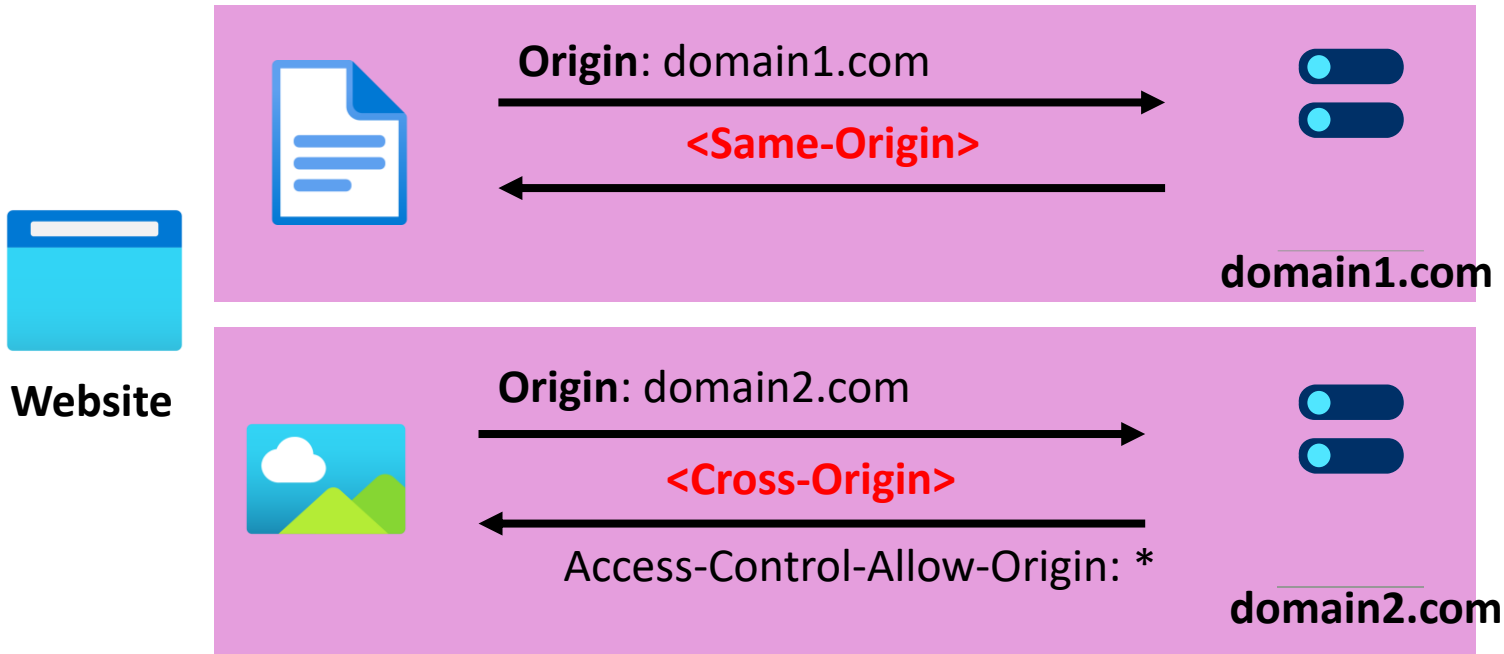
VPC Gateway Endpoint can not go cross-account

Does not have fine-grain permissions

There is no charge to use VPC Gateway Endpoints

Cross-Origin Resource Sharing (CORS)

Cross-Origin Resource Sharing (CORS) is an HTTP-header based mechanism that allows a server to indicate any other origins (domain, scheme, or port) than its own from which a browser should permit loading of resources



Access is controlled via HTTP headers

Request Headers

- Origin
- Access-Control-Request-Method
- Access-Control-Request-Headers

Response Headers

- Access-Control-Allow-Origin
- Access-Control-Allow-Credentials
- Access-Control-Expose-Headers
- Access-Control-Max-Age
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers

CORS restrict which websites may access data to be loaded onto its page

Amazon S3 CORS

Amazon S3 allows you to set CORS configuration to a S3 bucket with static website hosting so different origins can perform HTTP requests from your S3 Static website.

The CORS configuration can be in either **JSON** or XML

- The AWS Console only access JSON.

In real-world use cases, **you should not wildcard (allow all) origin** since this would negate the protection that CORS is supposed to serve.

```
[{
  "AllowedHeaders": ["*"],
  "AllowedMethods": ["PUT", "POST", "DELETE"],
  "AllowedOrigins": ["http://www.example1.com"],
  "ExposeHeaders": []
}, {
  "AllowedHeaders": ["*"],
  "AllowedMethods": ["PUT", "POST", "DELETE"],
  "AllowedOrigins": ["http://www.example2.com"],
  "ExposeHeaders": []
}, {
  "AllowedHeaders": [],
  "AllowedMethods": ["GET"],
  "AllowedOrigins": ["*"],
  "ExposeHeaders": []
}]
```

S3 Encryption Overview

Encryption In Transit

When data is encrypted by the sender and then decrypted the reciever.

Encryption-At-Rest

Client-Side Encryption (CSE)

when data is encrypted by the client and then sent to the server

The client has the key, the server will serve the encrypted file since it does not have the key to decrypt when data is requested

Server-Side Encryption (SSE)

when data is encrypted by the server

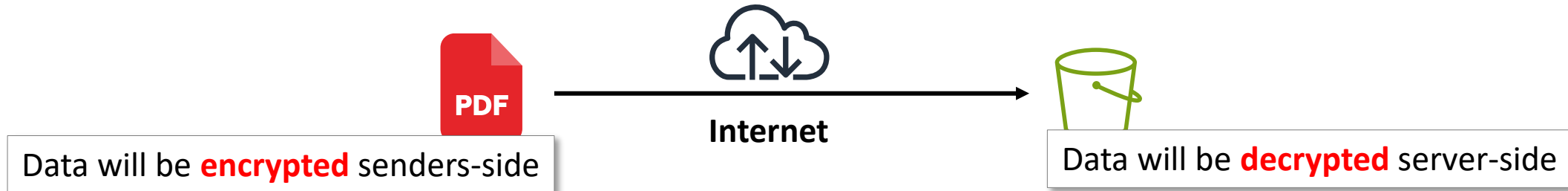
The server has the key to decrypt when data is requested

S3 – Encryption In Transit

Encryption In-Transit

Data that is secure when moving between locations Algorithms: **TLS, SSL**

This encryption ensures that data remains confidential and **cannot be intercepted or viewed** by unauthorized parties while in transit.



Transport Layer Security (TLS)

An encryption protocol for data integrity between two or more communicating computer application.

*TLS 1.0, 1.1 are deprecated. **TLS 1.2** and **TLS 1.3** is the current best practice*

Secure Sockets Layers (SSL)

An encryption protocol for data integrity between two or more communicating computer application

SSL 1.0, 2.0 and 3.0 are deprecated

S3 – Server-Side Encryption

Server-Side Encryption (SSE) is **enabled by default** for all new S3 objects.

SSE-S3

Amazon S3 manages the keys, encrypting objects using **AES-256 (Cipher Block Chaining – CBC)** algorithm

SSE-KMS

AWS Key Management Service (KMS) manages encryption keys. You can choose AWS-managed keys or manage customer master keys (CMKs) yourself through KMS.

SSE-C

You provide your own encryption keys, and Amazon S3 handles encryption and decryption using these customer-provided keys. You are responsible for managing the keys.

DSSE-KMS (Dual-layer Encryption)

In dual-layer encryption, data is encrypted client-side before being uploaded to S3, and then server-side encryption (SSE) is applied within S3. This is not a specific AWS service but a strategy some users may adopt.

Server-side encryption only encrypts the contents of an object, not its metadata

Amazon S3 – SSE-S3

What is SSE-S3?

- Amazon manages all encryption for objects stored in S3
- Each object is encrypted with a unique key
- Envelope encryption is used
- Keys are rotated automatically by Amazon
- By default, SSE-S3 is applied to all objects unless specified otherwise
- There are **no additional charges** for using SSE-S3
- Uses **AES-256-but Advanced Encryption Standard (AES-GCM)** for securing data

You can **explicitly set** SSE-S3 when uploading objects



```
aws s3api put-object \  
--bucket mybucket \  
--key myfile \  
--server-side-encryption AES256 \  
--body myfile.txt
```

By default, if no encryption configuration is specified, SSE-S3 is automatically applied



```
aws s3api put-object \  
--bucket mybucket \  
--key myfile \  
--body myfile.txt
```

Amazon S3 – SSE-KMS

SSE-KMS is when you use a KMS key managed by AWS.

- You first create a KMS managed key
- You choose the KMS key to encrypt your object
- KMS can automatically rotate keys
- KMS key policy controls who can decrypt using the key
- KMS can help meet regulatory compliance
- KMS keys have their own additional costs
- AWS KMS keys can be multi-region, allowing the same key to be used in different regions, not limited to the bucket's region.
- To upload with KMS you need kms:GenerateDataKey
- To download with KMS you need kms:Decrypt

Bucket Key can be set for SSE-KMS for improved performance

Using a KMS with **aws s3api**



```
aws s3api put-object \  
--bucket mybucket \  
--key example.txt \  
--body example.txt \  
--server-side-encryption "aws:kms" \  
--ssekms-key-id 1234abcd-12ab-34cd-56ef-1234567890ab
```

Using a KMS key with **aws s3**



```
aws s3 cp example.txt s3://mybucket/example.txt \  
--sse aws:kms \  
--sse-kms-key-id 1234abcd-12ab-34cd-56ef-1234567890ab
```

Amazon S3 – SSE-C

SSE-C is when you provide your own encryption key that Amazon S3 then uses to apply AES-256 encryption to your data.

- You need to provide the encryption key every time you retrieve objects
 - The encryption key you upload is removed from Amazon S3 memory after each request
- There is no additional charge to use SSE-C
- Amazon S3 will store a randomly salted Hash-based Message Authentication Code (HMAC) of your encryption key to validate future requests
- Presigned URLs support SSE-C

With bucket versioning, different object versions can be encrypted with different keys

you manage encryption keys on the client side, you manage any additional safeguards, such as key rotation, on the client side.

```
$BASE64_ENCODED_KEY=(openssl rand 32 | base64)
```

```
aws s3api put-object \  
--bucket mybucket \  
--key myfile \  
--body file://myfile.txt \  
--sse-customer-algorithm AES256 \  
--sse-customer-key $BASE64_ENCODED_KEY \  
--sse-customer-key-md5 `echo -n \  
$BASE64_ENCODED_KEY | base64 --decode | \  
md5sum | awk '{print $1}' | base64`
```

Amazon S3 – DSSE-KMS

DSSE (Dual-layer server-side encryption) KMS. It's SSE-KMS with **the inclusion of client-side encryption.**

It would be more accurate to call it CSE-KMS

- With DSSE-KMS, data is encrypted twice
- The key used for client-side encryption comes from KMS
- There are additional charges for DSSE and KMS keys

```
aws s3api put-object \  
  --bucket mybucket \  
  --key myfile \  
  --server-side-encryption aws:kms:dsse \  
  --ssekms-key-id "1234abcd-12ab-34cd-56ef-1234567890ab" \  
  --body filepath
```

Encrypt

- client-side requests AWS KMS to generate a data encryption key (DEK) using the CMK.
- KMS sends two versions of the DEK to you: a plaintext version and an encrypted version
- You use the plaintext DEK to encrypt your data locally and then discard it from memory.
- encrypted version of the DEK is stored alongside the encrypted data in Amazon s3

Decrypt

- you retrieve the encrypted data and the associated encrypted DEK
- You send the encrypted DEK to AWS KMS, which decrypts it using the corresponding CMK and returns the plaintext DEK.
- use this plaintext DEK to decrypt the data locally and then discard the DEK from memory

S3 Bucket Key

When you use SSE-KMS, an individual data key is used on every object request
In this case S3, has to call AWS KMS every time a request is made.
KMS charges on the number of requests and so this charge can add up

S3 Bucket key lets **you generate a short-lived bucket-level key** from AWS Key that is temporarily stored in S3

- This will **reduce request costs by up to 99%**
 - This will decrease request traffic and improve overall performance
-
- A unique bucket-level key is generated for each requester
 - You can enable Bucket key at the bucket level to be applied to all new objects
 - You can enable Bucket key at the object level for only specific objects
 - S3 Bucket key can be enabled for SSE-S3 and SSE-KMS

```
aws s3api put-bucket-encryption \
--bucket mybucket \
--server-side-encryption-configuration '{
  "Rules": [{
    "ApplyServerSideEncryptionByDefault": {
      "SSEAlgorithm": "aws:kms",
      "KMSMasterKeyID": "alias/your-kms-key-alias"
    },
    "BucketKeyEnabled": true
  }]
}'
```

S3 – Client-Side Encryption

Client-Side Encryption is when you encrypt your own files before uploading them to S3. This provides a guarantee that AWS and no third-party can decrypt your data.

Various AWS SDK's have built-in code to make it easy to encrypt your data

Ruby SDK you can **supply an encryption key** to the S3 Client, and it will automatically encrypt and decrypt server side.

Supplying no key will result in the cipher text

```
require 'openssl'

s3 = Aws::S3::Encryption::Client.new({
  encryption_key: OpenSSL::PKey::RSA.new(1024)
})

attrs = { bucket: 'mybucket', key: 'secret' }

# round-trip an object, encrypted/decrypted
s3.put_object(attrs.merge({body: 'secretdata'}))
s3.get_object(attrs).body.read
#=> 'secretdata'

# new client gets cipher text
Aws::S3::Client.new.get_object(attrs).body.read
#=> "... cipher text ..."
```

S3 – Data Consistency

What is data consistency?

When data is stored in multiple locations, **data consistency** ensures the values are the same across those locations

Strongly Consistent

- Every read (query) returns the most recent write.
- Guarantees **no stale data**.
- Read options might be slightly slower but always accurate.

We will never return your old data. But you will have to wait at least 2 seconds for the query to return

Eventually Consistent

- Data may be outdated on immediate read after a write.
- If you wait, the data will **eventually reflect the latest state**.
- Faster read response time, but not guaranteed to be up-to-date.

We are giving you whatever data is currently in the database, you may get new data or old data, but if you wait a little bit longer it will generally be up to date.

Amazon S3 offers **strong consistency** for all read, write, and delete operations.

Prior to Jan 2020, S3 did not offer strong consistency for all s3 operations

S3 – Object Replication Overview



Amazon S3 Object Replication is when you make
(copies) of objects into other buckets

- **Data Redundancy and Durability:** Enhances data protection by storing copies in different locations.
- **Compliance and Data Residency:** Meets legal and regulatory data storage requirements in specific regions.
- **Improved Accessibility and Latency:** Reduces latency by replicating data closer to end-users.
- **Operational Flexibility:** Separates production, backup, and development data sets for various operational needs.
- **Efficient Data Processing:** Facilitates parallel data processing in regions closer to AWS resources.
- **Disaster Recovery and Business Continuity:** Ensures business continuity with backup copies in separate locations.
- **Optimized Costs:** Costs can be reduced by replicating to regions with lower storage pricing.
- **Cross-Account Replication:** Allows secure data sharing between different AWS accounts.
- **Versioning and Change Tracking:** Maintains object versions in different buckets for easy rollback and tracking.
- **Automated Backups and Archiving:** Simplifies and updates backup and archiving processes automatically.

S3 – Object Replication

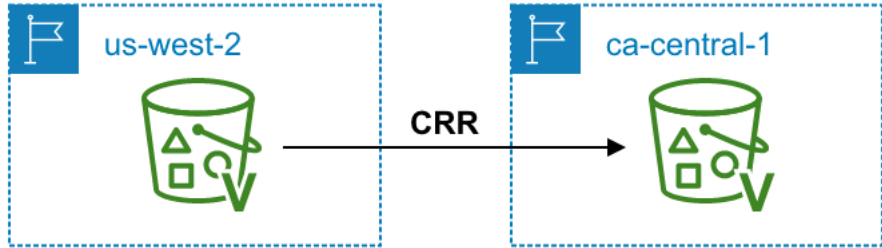
Object Replication can help you do the following

- Replicate objects while retaining metadata
- Replicate objects into different storage classes
- Maintain object copies under different ownership
- Keep objects stored over multiple AWS Regions
- Replicate objects within 15 minutes
- Sync buckets, replicate existing objects, and replicate previously failed or replicated objects
- Replicate objects and fail over to a bucket in another AWS Region

Replication options in S3

- **Cross-Region Replication (CRR)**: Replicates objects between regions (live)
- **Same-Region Replication (SRR)**: Replicates within the same region (live)
- **Bi-Directional Replication**: Two-way replication between buckets (live)
- **S3 Batch Replication**: On-demand replication of existing objects

S3 – Cross Region Replication



Cross-Region Replication (CRR) is when bucket objects are replicated to **a bucket in a different region**.

Reason for using Cross Region Replication

- Provides higher durability and potential disaster recovery for objects
- Some companies need objects to exist in multiple regions for regulatory compliance
- Accessing objects in the nearest region will result in minimized latency

Considerations

- **You must have S3 versioning turned on** both the **source** and **destination** buckets.
- You can have CRR replicated to another AWS account Cross-Account Cross Region Replication (CA-CRR)
- It will replicate new or updated objects. Objects before the replication rule will not be replicated

S3 – Cross Region Replication

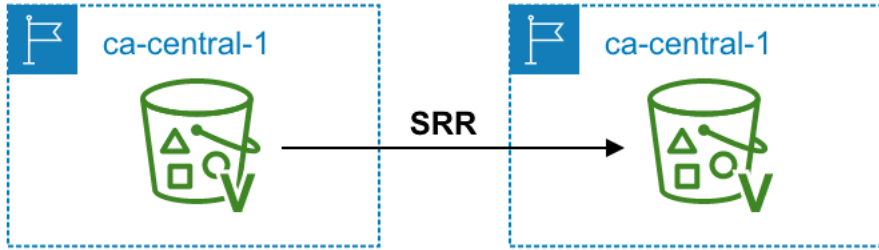
We create a new **replication rule**:

```
aws s3api put-bucket-replication \  
--bucket source-bucket \  
--replication-configuration file://replication.json
```

We **configure our rule** for the destination bucket (which happens to be in another region).

```
{  
  "Role": "arn:aws:iam::123456789012:role/my-s3-repl-role",  
  "Rules": [{  
    "ID": "CRRRule",  
    "Priority": 1,  
    "Status": "Enabled",  
    "Filter": { "Prefix": "" },  
    "Destination": {  
      "Bucket": "arn:aws:s3:::destination-bucket",  
      "StorageClass": "STANDARD"  
    }  
  }]  
}
```

S3 – Same Region Replication



Same-Region Replication (SRR) is when bucket objects are replicated to **a bucket in the same AWS region.**

Reason for using Same Region Replication

- Aggerating logs from multiple buckets into a single bucket
- Replication of objects for different workload environments eg. Development, QnA, Production, Staging
- Sovereignty laws require you to keep data in a specific region.

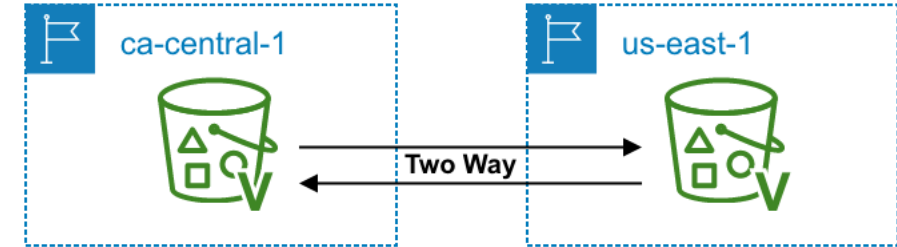
Considerations

- You must have **S3 versioning** turned on both the **source** and **destination** buckets.
- You can have SRR replicated to another AWS account Cross-Account Same-Region Replication (CA-SRR)
- It will replicate new or updated objects. Objects before the replication rule will not be replicated

The replication rule for SSR is identical to the CRR replication rule.
The destination bucket points to a bucket in the same region.

S3 – Bi-Directional Replication

Bi-Directional Replication aka (two-way) replication bucket objects replicate **data in both directions between two buckets**



Reason for using Bi-Directional Replication

- You need to fail over from Bucket A to Bucket B due to an issue with Bucket A
 - When Bucket A issue is resolved, you want to have new content in Bucket B back in Bucket A
 - Then you switch back to Bucket A as your primary bucket.

Considerations

- You can use either the same or a different region
- You can replicate across accounts.
- You can use a **S3 Multi-region Access Point** with buckets in different regions

The replication rule for SSR is identical and CRR, but you need to create two replication rules for each direction

S3 – Batch Replication

Batch Replication replicates existing objects to different buckets **as an on-demand option**

Reason for using Batch Replication

- Replicate existing objects that were added to the bucket before Same-Region Replication (SRR) or Cross-Region Replication (CRR) rules were applied
- Replicate objects that previously failed to replicate due to errors
- Replicate objects that were already replicated (e.g., for compliance or additional backups)
- Replicate replicas of objects created by an existing replication rule
- Perform replication on an as-needed basis, such as for specific projects or compliance requirements



Batch replication is performed via **S3 Batch Operations**.

Use the AWS CLI command `aws s3control create-job` with the `S3ReplicateObject` operation to define and manage batch replication tasks efficiently through S3 Batch Operations

S3 – Replication Rules Configuration



Replication Time Control (RTC)

replicates 99.99% of new objects within 15 minutes and includes replication metrics. Additional fees will apply.

Replication metrics

monitor the total number and size of objects that are pending replication, and the maximum replication time to the destination Region. You can also view and diagnose replication failures. CloudWatch metrics fees apply.

Delete marker replication

Delete markers created by S3 delete operations will be replicated.

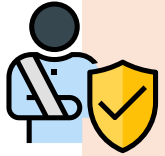
Delete markers created by lifecycle rules are not replicated

- a delete marker is a placeholder (or marker) used to indicate that an object has been deleted.

- ☐ **Replication Time Control (RTC)**
Replication Time Control replicates 99.99% of new objects within 15 minutes and includes replication metrics. Additional fees will apply. [Learn more](#)
- ☐ **Replication metrics**
With replication metrics, you can monitor the total number and size of objects that are pending replication, and the maximum replication time to the destination Region. You can also view and diagnose replication failures. CloudWatch metrics fees apply. [Learn more](#) or see [Amazon CloudWatch pricing](#)
- ☐ **Delete marker replication**
Delete markers created by S3 delete operations will be replicated. Delete markers created by lifecycle rules are not replicated. [Learn more](#)
- ☐ **Replica modification sync**
Replicate metadata changes made to replicas in the destination Region.

S3 Versioning

S3 Versioning allows you to store multiple versions of S3 objects.



With versioning, you can **recover more easily** from unintended user actions and application failures



Versioning-enabled buckets can help you recover objects from **accidental deletion or overwrite.**

- Store all versions of an object in S3 at the same object key address
- By default, S3 Versioning is disabled on buckets, and you must explicitly enable it
- Once enabled, it cannot be disabled, only suspended on the bucket
- Fully integrates with S3 Lifecycle rules
- MFA Delete feature provides extra protection against deletion of your data

Buckets can be in three states:

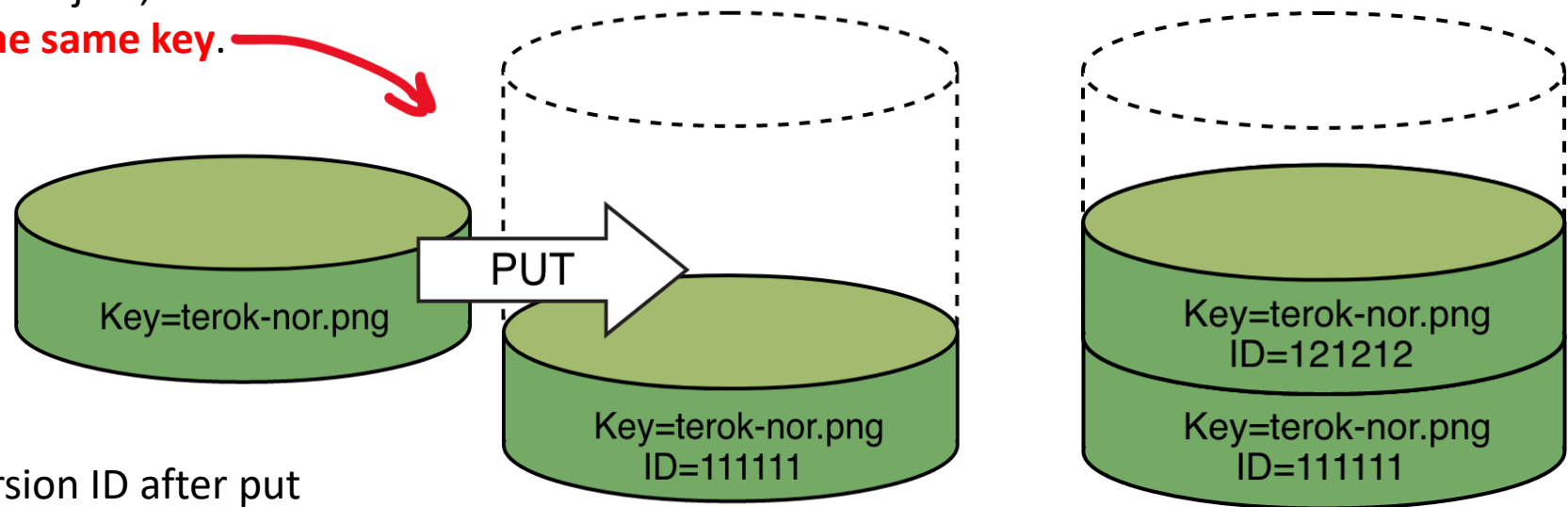
1. Unversioned (default)
2. Versioned
3. Versioned Suspended

S3 Versioning – Adding Object Versions

When you enable S3 Versioning, all objects will generate a **Unique Version ID**

x-amz-version-id: 3/L4kqtJlcpXroDTDmJ+rmSpXd3dlbrHY

When you update an S3 object, it will store both objects **at the same key**.



The CLI will return the Version ID after put

```
aws s3api put-object  
--bucket my-bucket \  
--key my-object \  
--body file.txt
```

```
{  
  "ETag": "\"6805f2cfc46c0f04559748bb039d69ae\"",  
  "VersionId": "qL8nFgEUV6w7VTpD6E.XYZ.UZKL07Ho"  
}
```

S3 Versioning – Listing Object Versions

We can list object versions for a specific object using the AWS CLI

```
aws s3api list-object-versions \  
--bucket my-bucket \  
--prefix index.html
```



We can get back the Versions (and the delete markers) for an object.

```
{  
  "DeleteMarkers": [  
    {  
      "Owner": {  
        "DisplayName": "my-username",  
        "ID": "7009a8971cd660687538875e7c86c5b67..."  
      },  
      "IsLatest": true,  
      "VersionId": "B2VsEK5saUNNHKc0AJj7hIE86RozToyq",  
      "Key": "index.html",  
      "LastModified": "2015-11-10T00:57:03.000Z"  
    },  
    // ...  
  ],  
  "Versions": [  
    {  
      "LastModified": "2015-11-10T00:20:11.000Z",  
      "VersionId": "Rb_l2T8UHDkFEwCgJjhlqP0ZC0qJ.vpD",  
      "ETag": "\"0622528de826c0df5db1258a23b80be5\"",  
      "StorageClass": "STANDARD",  
      "Key": "index.html",  
      "Owner": {  
        "DisplayName": "my-username",  
        "ID": "7009a8971cd660687538875e7..."  
      },  
      "IsLatest": false,  
      "Size": 38  
    },  
    // ...  
  ]  
}
```

S3 Versioning – Get Object Versions

When you get GET an S3 object, it **will always return the most current version** of the object.

```
aws s3api get-object \  
--bucket example-bucket \  
--key example.txt \  
downloaded-example.txt
```



When you want a **specific version of an object**, you need to specify the version id.

```
aws s3api get-object \  
--bucket example-bucket \  
--key example.txt \  
--version-id 3_L4kqYxitTVAiv1MR8K4xJ_AKVUxZygP \  
downloaded-example.txt
```



Retrieving metadata will always return the current version. You need to specify the version object for versioned metadata.

S3 Versioning – Deleting Object Versions

When you delete an object without specifying a version, it makes the current version a **noncurrent version**, it will then add a Delete Marker as the current version.

What is a Delete Marker

A Delete marker is a placeholder for a versioned object

- It has a key and version ID, just like a versioned object
- It doesn't have data associated with it.
- It isn't associated with an access control list (ACL) value.
- It takes up space and incurs storage charge (filesize is the size of the key name)
- If the current version is a delete marker, it will return a 404 and response header **x-amz-delete-marker: true**
- If a specific version is specified and it's a delete marker, you'll get a 405 and response header x-amz-delete-marker and Last-Modified

```
aws s3api delete-object \  
--bucket example-bucket \  
--key example.txt \  
--version-id 3_L4kqYxitTVAiv1MR8K4xJ_AKVUxZygP
```



If you want to permanently delete an object version, you need specify its **--version-id**

S3 Versioning – Restoring Object Versions

You can restore previous versions in two ways:

1. Copy a previous version of the object into the same bucket

The copied object becomes the current version of that object, and all object versions are preserved.

2. Permanently delete the current version of the object

When you delete the current object version, you, in effect, turn the previous version into the current version of that object

S3 Versioning – MFA Delete

MFA Delete is an optional security precaution for S3 Versioning that requires you to use multi-factor before you can delete an object.

MFA Delete can only be enabled under these conditions

1. Can only be enabled via the AWS API eg. AWS CLI
2. The bucket must have **versioning turned on**



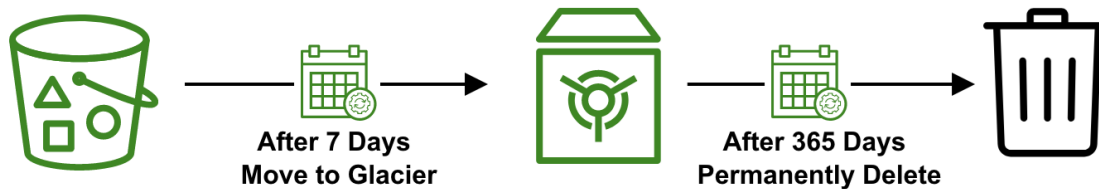
```
aws s3api put-bucket-versioning \  
--bucket my-bucket \  
--versioning-configuration Status=Enabled,MFADelete=Enabled \  
--mfa "your-mfa-serial-number mfa-code"
```

Only the bucket owner logged in as Root User can DELETE objects from bucket

Introduction to S3 Lifecycle

S3 Lifecycle allows you to automate the storage class changes, archival, or deletion of objects

- Can be used together with **versioning**
- Can be applied to both **current** and **previous** versions



There are two types of Actions

- Transition Actions
- Expiring Actions

Lifecycle Rule Actions

- Move current versions of objects between storage classes
- Move noncurrent versions of objects between storage classes
- Expire current versions of objects
- Permanently delete noncurrent versions of objects
- Delete expired object delete markers, or incomplete multipart uploads

Lifecycle Filters

- Filter based on prefix
- Filter based on Object tags
- Filter based on Object size (min and/or max)

S3 Lifecycle – Transitioning Objects

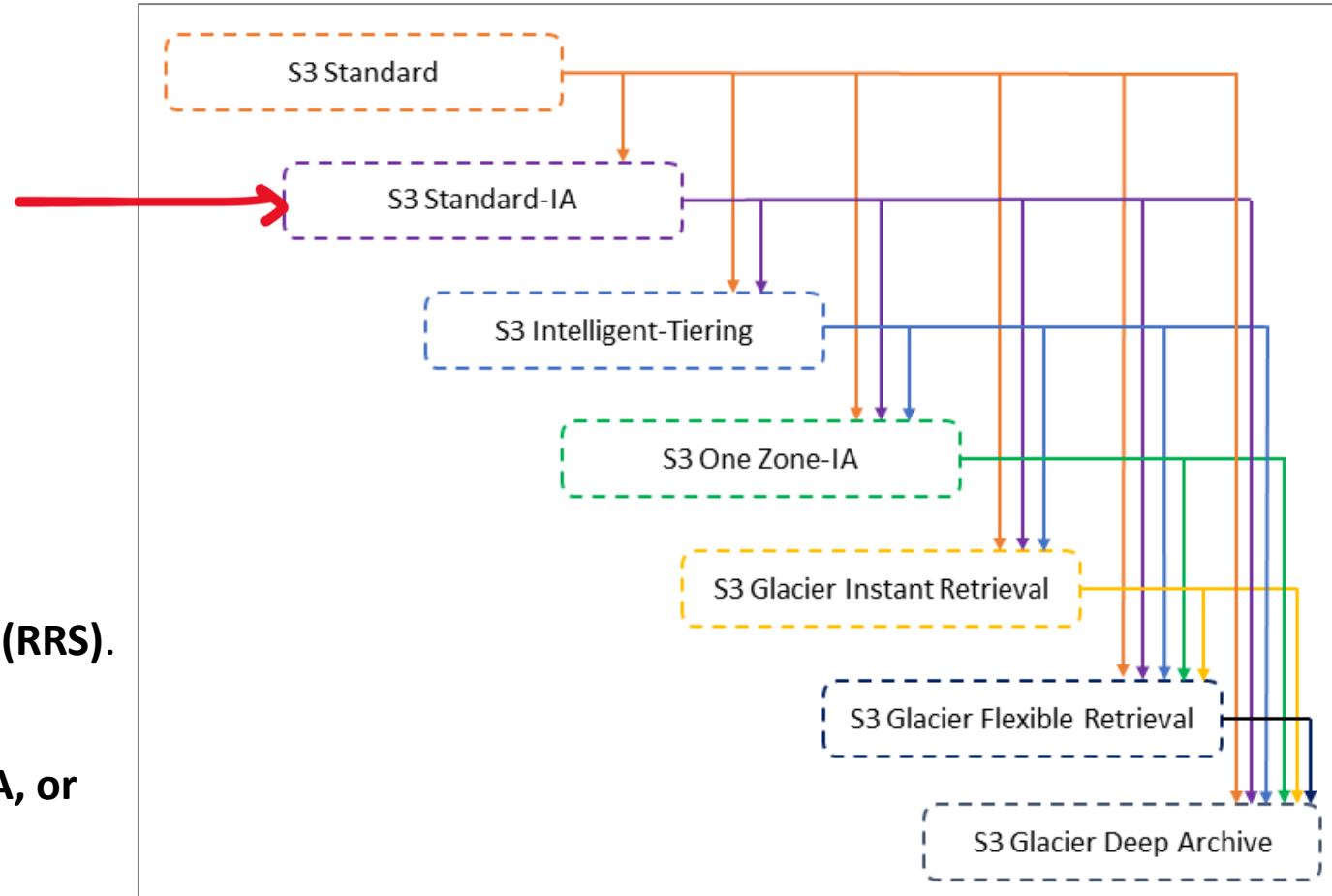
You can define rules to transition objects from one storage class to another to **save on storage costs**

Amazon S3 has a **waterfall model** when transitioning between storage classes with S3 Lifecycle:

- You can move things down
- You can't move them back up

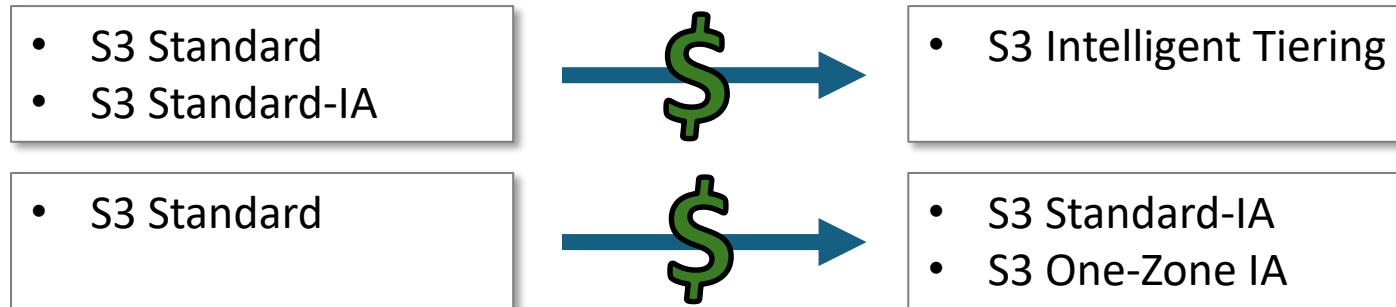
You can't transition from the following:

- Any storage class to the **Standard**
- Any storage class to **Reduced Redundancy Storage (RRS)**.
- The **Intelligent-Tiering** to the **Standard-IA**.
- The **One Zone-IA** to **Intelligent-Tiering**, **Standard-IA**, or **Glacier Instant Retrieval**

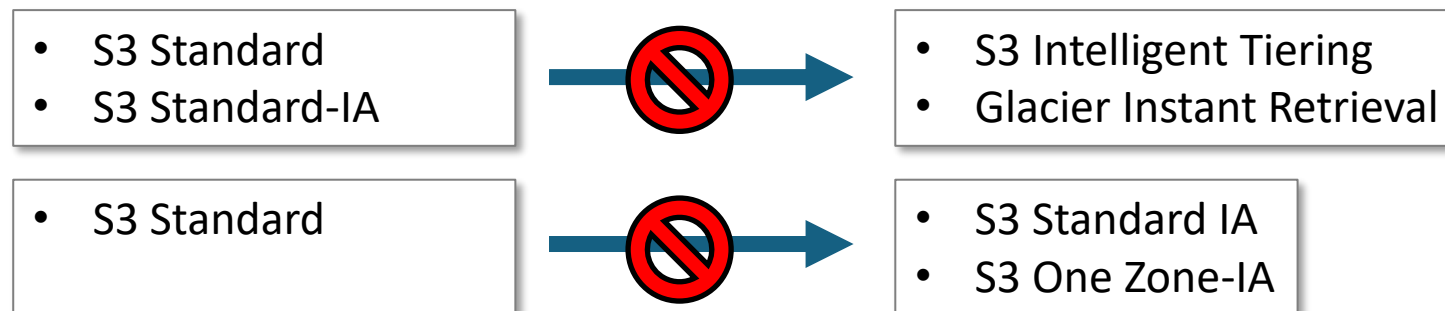


S3 Lifecycle – Transitioning Objects

Larger Objects —There's a cost benefit for these transitions



Smaller Objects —For objects less than 128 KiB, these transitions are not allowed



Use object filters (e.g., by file size: min and max) to better manage lifecycle transitions

S3 Lifecycle – Transitioning Objects

Minimum Days — when you must store objects in S3 for 30 days before transitioning

- S3 Standard



Must wait
30 days



- S3 Standard-IA

- S3 Standard
- S3 Standard-IA
- S3 Intelligent Tiering



Must wait
30 days



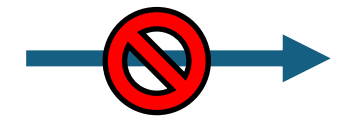
- S3 One Zone-IA

Minimum Storage Charge — these storage classes have a minimum charge of 30 days

- S3 Standard-IA
- S3 One Zone-IA



30 day
charge



Less than 30 days

- S3 Glacier Flexible Retrieval
- Glacier Deep Archive

Because you have minimum 30-day charge for Standard-IA and One Zone-IA you **can't transition less than 30 days**.

S3 Lifecycle – Expiring Objects

Amazon can expire objects based on three bucket states:

- **Non-versioned bucket** — Delete the objects
- **Versioned-enabled bucket** — Adds a delete marker with a unique version
- **Versioned-suspended bucket** — Creates a delete marker with a null version ID

Minimum Storage duration

An S3 lifecycle expiration rule for objects that require minimum storage will still result in the full charge

- 30 Day Charge — S3 Standard-IA or S3 Zone-IA
- 90 Day Charge — S3 Glacier Flexible Retrieval
- 180 Day Charge — S3 Glacier Deep Archive

S3 Lifecycle – Configuration Considerations

Propagation delay

Expect a delay of a few minutes before the configuration fully takes effect

Disabling or deleting Lifecycle rules

S3 stops scheduling new objects for deletion or transition after a small delay.

Any objects that were already scheduled are unscheduled and are not deleted or transitioned.

Existing and new objects

configuration rules apply to both existing objects and objects that you add later.

Changes in billing

changes in billing happen as soon as the Lifecycle configuration rule is satisfied, even if the action has not yet been taken

S3 Lifecycle – Anatomy of a Lifecycle Policy

Lifecycle Policies via the AWS CLI can either be *XML or JSON

You can have up to 1000 rules.

ID uniquely identifies a rule. 256-character limit

Status can be either Disabled or Enabled

Filter allows you to filter specific objects

Transition — actions that let you perform transitions

Expiration — actions that let you perform expirations

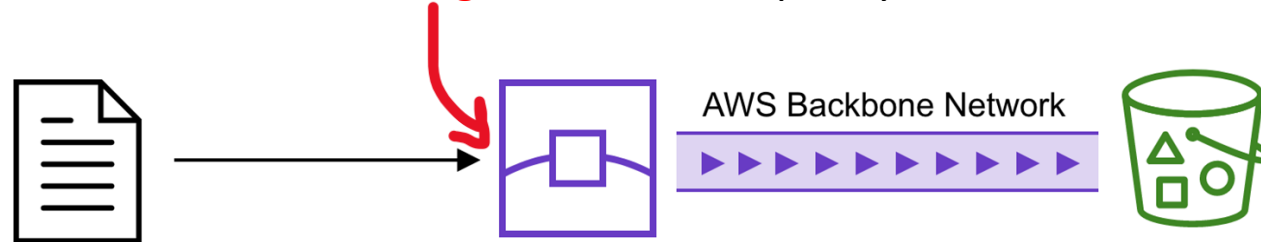
```
{
  "Rules": [
    {
      "ID": "ExampleRule",
      "Status": "Enabled",
      "Filter": {
        "Prefix": "documents/"
      },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        }
      ],
      "Expiration": {"Days": 365 }
    }
  ]
}
```

```
aws s3api put-bucket-lifecycle-configuration \
  --bucket mybucket \
  --lifecycle-configuration file://lifecycle.json
```

Introduction to S3 Transfer Acceleration

S3 Transfer Acceleration is a bucket-level feature that provides fast and secure transfer of files **over long distances** between your end users and an S3 bucket.

Utilizes **CloudFront's** distributed **Edge Locations** to quickly enter the Amazon Global Network



Instead of uploading to your bucket, users use a **distinct endpoint** to route to an edge location

- <https://s3-accelerate.amazonaws.com>
- <https://s3-accelerate.dualstack.amazonaws.com>

- only supported on **virtual-hosted style requests**
- Buckets cannot contain periods and must be DNS-compliant
- It can take 20 minutes after Transfer Acceleration is enabled

S3 Transfer Acceleration – How to use

```
aws s3api put-bucket-accelerate-configuration \  
--bucket mybucket \  
--accelerate-configuration Status=Enabled
```

Enable Transfer Acceleration on the bucket

```
# ensure virtual style  
aws configure set s3.addressing_style virtual
```

Ensure you are using **virtual-hosted style** requests

```
# supply s3 transfer acceleration endpoint  
aws s3 cp file.txt s3://bucketname/keyname \  
--region ca-central-1 \  
--endpoint-url https://s3-accelerate.amazonaws.com
```

You'll use the **--endpoint-url** flag to ensure the S3 Transfer Acceleration endpoint is being used.

You can *optionally* tell the AWS CLI to always use the S3 Transfer Acceleration endpoint

```
aws configure set default.s3.use_accelerate_endpoint true
```

S3 – Presigned URLs

S3 Presigned Urls provides **temporary access to upload or download object data** via URL.



Presigned URLs are commonly used to provide access to **private objects**.

You can use **AWS CLI** or AWS SDK to generate a Presigned URL.

The expires is in **seconds**

```
aws s3 presign s3://mybucket/myobject \  
--expires-in 300
```

```
https://mybucket.s3.amazonaws.com/myobject?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=YOUR_AWS_ACCESS_KEY%2F20231125%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-Date=20231125T123456Z&X-Amz-Expires=300&X-Amz-SignedHeaders=host&X-Amz-Signature=GENERATED_SIGNATURE
```

This is the presigned url

S3 – Anatomy of a Presigned URLs

```
https://mybucket.s3.amazonaws.com/myobject  
?X-Amz-Algorithm=AWS4-HMAC-SHA256  
&X-Amz-Credential=YOUR_AWS_ACCESS_KEY%2F20231125%2Fus-east-1...  
&X-Amz-Date=20231125T123456Z  
&X-Amz-Expires=300  
&X-Amz-SignedHeaders=host  
&X-Amz-Signature=GENERATED_SIGNATURE
```

- **X-Amz-Algorithm** specifies the signing algorithm, typically AWS4-HMAC-SHA256.
- **X-Amz-Credential** includes your AWS access key and the scope of the signature.
- **X-Amz-Date** is the timestamp of when the signature was created.
- **X-Amz-Expires** defines the duration for which the URL is valid, in this case, 300 seconds.
- **X-Amz-SignedHeaders** indicates which headers are part of the signing process.
- **X-Amz-Signature** the actual signature calculated based on your AWS secret access key, the string to sign, and the signing key.

S3 – Access Points



S3 Access Points simplify managing data access at scale for shared datasets in S3.

S3 Access Points are **named network endpoints that are attached to buckets** that you can use to perform S3 object operations eg. Get, Put

Each access point has:

- Distinct **permissions** via an Access Point Policy
- Distinct **network controls**
- Distinct block public access

Network Origin

- Internet – requests can come from the Internet.
- VPC – all requests must come from specified VPC

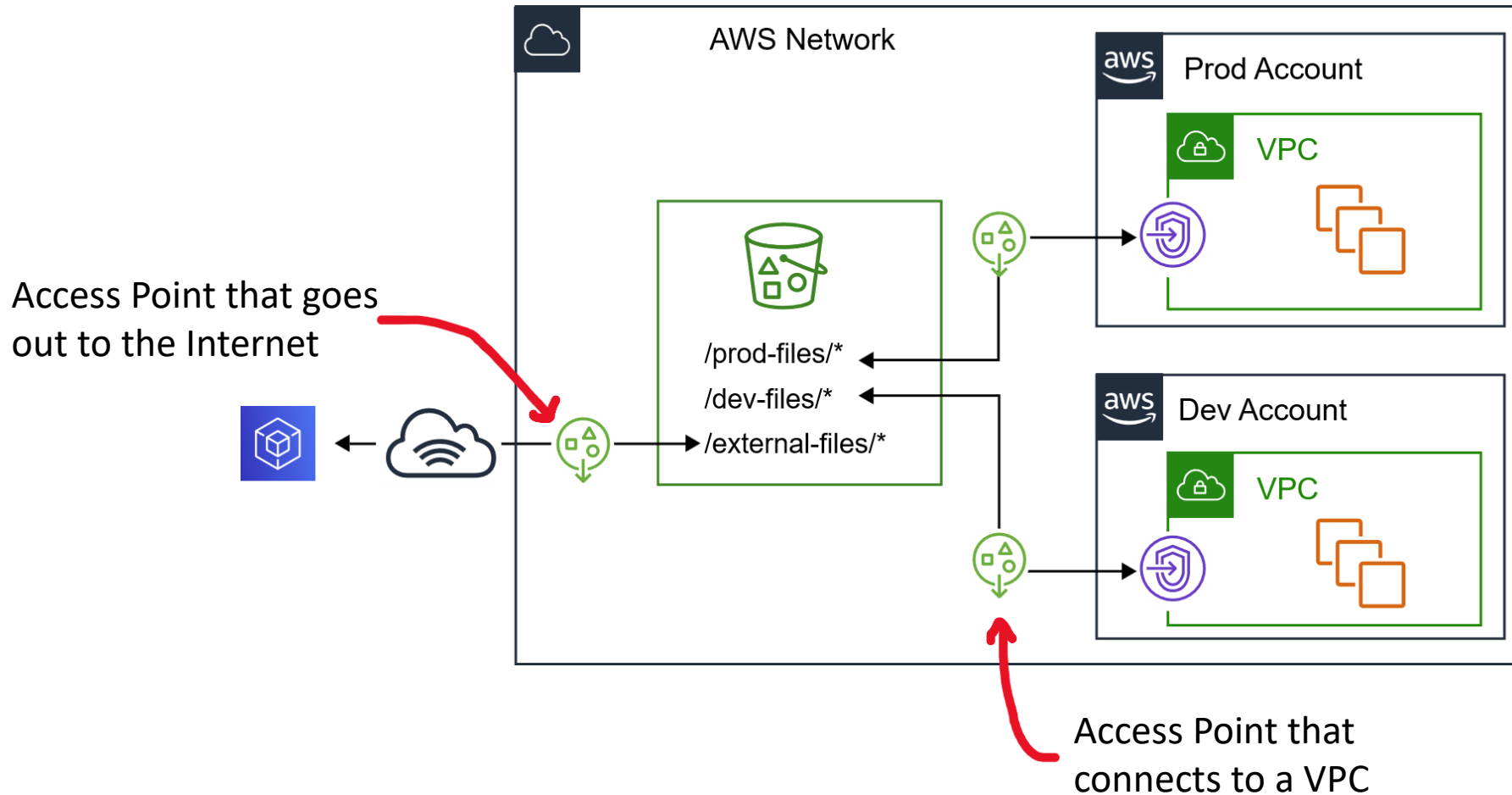
S3 Access Point Policy allows you to write permissions for a bucket alongside your bucket policy.

An Access Point Policy helps to **move specific and complex access configuration out of your bucket policy**, keeping your bucket policy simple and easy to read.

```
{
  "Version": "2012-10-27",
  "Statement": [
    {
      "Principal": "*",
      "Action": ["s3:GetObject"],
      "Effect": "Allow",
      "Resource": [
        "arn:aws:s3:::deep-space-nine/*",
        "arn:aws:s3:us-east-1:123456789012:accesspoint/my-access-point/prod/*"
      ]
    }
  ]
}
```

S3 – Access Points

Instead of creating a Bucket Policy you can create multiple access points that allow you to apply policies to parts of your bucket's content

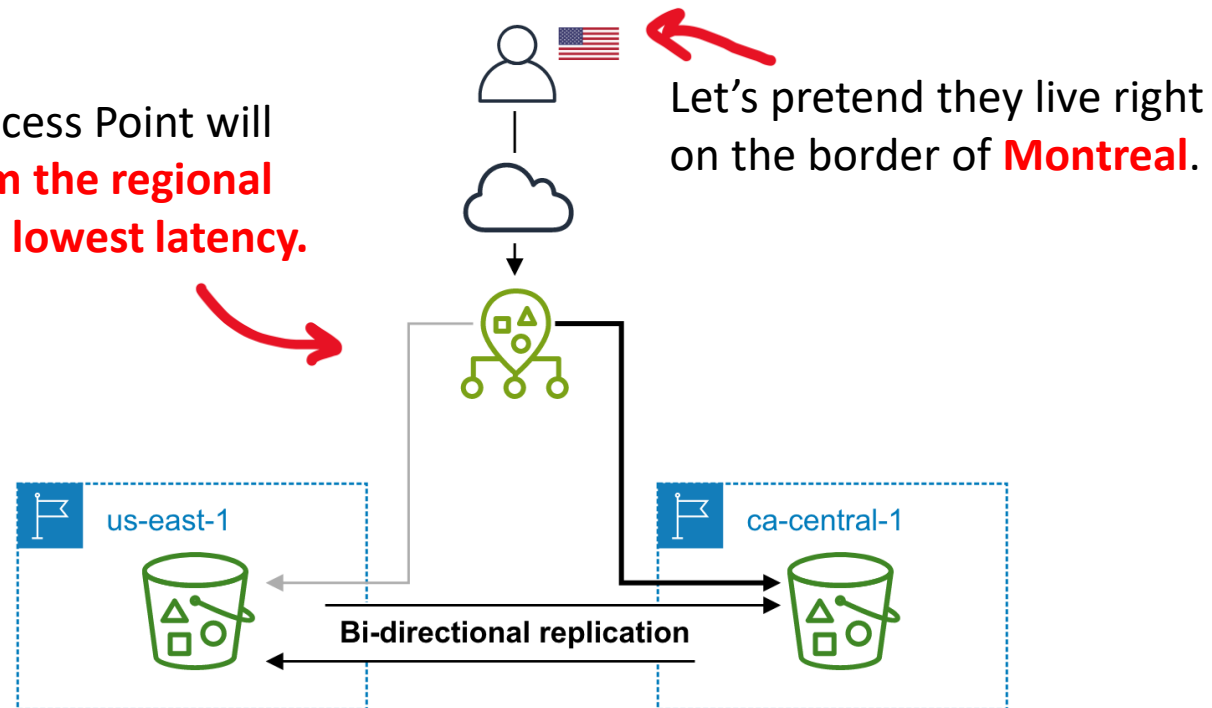


S3 – Multi-Region Access Points



Multi-Region Access Points is a global endpoint to route requests to multiple buckets residing in different regions.

Multi-Region Access Point will **return data from the regional bucket with the lowest latency.**



- **AWS Global Accelerator** is used to route to the closest bucket
 - Requests are accelerated over the internet, VPC or PrivateLink
- **S3 Replication Rules** can be used to synchronize objects to the regional buckets

S3 – Object Lambda Access Points



S3 Object Lambda Access Points allow you to **transform the output requests of S3 objects** when you want to present data differently.

S3 Object Lambda Access Points only operate on the outputted objects. The original objects in the S3 bucket remain unmodified

- ☒ **GetObject**
Retrieves objects from Amazon S3
- ☒ **HeadObject**
Retrieves metadata from an S3 object
- ☐ **ListObjects**
Returns up to 1000 objects in an S3 bucket
- ☐ **ListObjectsV2**
Returns up to 1000 objects in an S3 bucket

S3 Object Lambda can be performed on the S3 Operations

- HEAD — information about the S3 object, but not the object contents itself
- GET — an s3 object including its contents.
- LIST — a list of s3 objects

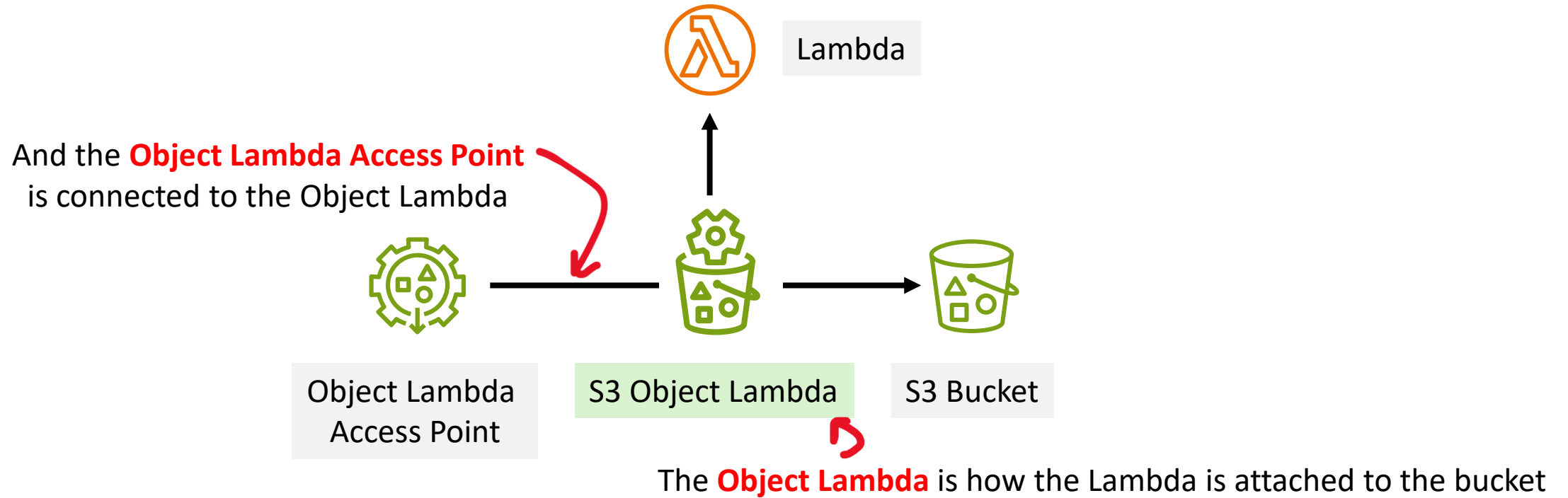


An Amazon Lambda Function is attached to an S3 Bucket via the **Object Lambda Access Point**



Multiple transformations can be configured per Object Lambda Access Point

S3 – Object Lambda Access Points



Mountpoint for Amazon S3

Mountpoint for Amazon S3 allows you to **mount an S3 bucket** to your **Linux** local file system.

Mountpoint is an open-source client that you install on your Linux OS and provides high-throughput access to objects with basic file-system operations

Mountpoint **can**:

- read files up to 5 TB in size
- list and read existing files
- create new files

Mounting **cannot**/does not

- modify existing files
- delete directories
- support symbolic links
- support file locking

Mountpoint is ideal for apps that don't need all the features of a shared file system and POSIX-style permissions but require Amazon S3's elastic throughput to read and write large S3 datasets.

It can be used in the following Storage Classes

- **S3 Standard**
- ~~S3 Intelligent Tiering~~
- **S3 Standard IA**
- **S3 One-Zone IA**
- **Reduced Redundancy Storage (RRS)**
- **S3 Glacier Instant Retrieval**
- ~~S3 Glacier Flexible Retrieval~~
- ~~S3 Glacier Deep Archive~~
- ~~S3 Intelligent Tiering Archive Access~~
- ~~S3 Intelligent Tiering Deep Archive Access~~

Mountpoint for Amazon S3

Installing Mountpoint (*RPM example*)

```
wget https://s3.amazonaws.com/mountpoint-s3-release/latest/x86_64/mount-s3.rpm  
sudo yum install ./mount-s3.rpm  
mount-s3 --version
```

Using Mountpoint

```
mkdir ~/mnt  
mount-s3 mybucket ~/mnt  
cd mnt  
# perform basic operations  
# eg. cat, ls, pwd  
_____  
umount ~/mnt  
_____
```

- Create a folder
- Mount the bucket to the folder
- Go into the folder
- Perform basic filesystem operations
- Unmount when you're done

Archived Objects

Archived Objects are rarely-access objects Amazon S3 that **cannot be accessed in real-time** in exchange for a **reduced storage-cost**.

There are two ways to archive objects:

1. Archive Storage Classes

- When you know your access patterns
- Requires **manual** invention to move data
- Lower costs archive storage costs



S3 Glacier Flexible Retrieval

- Minutes to hours



S3 Glacier Deep Archive

- +12 hours

2. Archive Access Tiers

- When you don't know your access pattern
- **Automatically** moves data
- Slightly higher cost than Archive Storage Classes



S3 Intelligent-Tiering Archive Access tier

- Within minutes



S3 Intelligent-Tiering Deep Archive Access

- 12+ hours

Restoring Archived Objects

Archived objects are not immediately accessible for the following archive storage classes:



- S3 Glacier Flexible Retrieval
- S3 Glacier Deep Archive
- S3 Intelligent-Tiering Archive Access Tier
- S3 Intelligent-Tiering Deep Archive Access Tier

Restoring from **S3 Glacier Flexible Retrieval** or **S3 Glacier Deep Archive**

Temporary Copy

You must first report a copy of the object to its S3 bucket for a specified number of days

```
aws s3api restore-object \  
--bucket mybucket\  
--key myfile \  
--restore-request Days=25,GlacierJobParameters={"Tier"="Standard"}
```

Permanent Copy

To permanently restore the temporary object, make a copy, and it will remain permanently

```
aws s3 cp \  
s3://mybucket/myfile \  
s3://mybucket/myfilecopy
```



When you restore an archived object from S3 Glacier, you pay for both the archived object and the copy that you restored temporarily.

Restoring Archived Objects

Restoring from **S3 Intelligent-Tiering Archive** or **S3 Intelligent-Tiering Deep Archive**

Restore Copy

You initiate a restore object, and then you must wait until the object is moved into the **S3 Intelligent Frequent Access tier**.

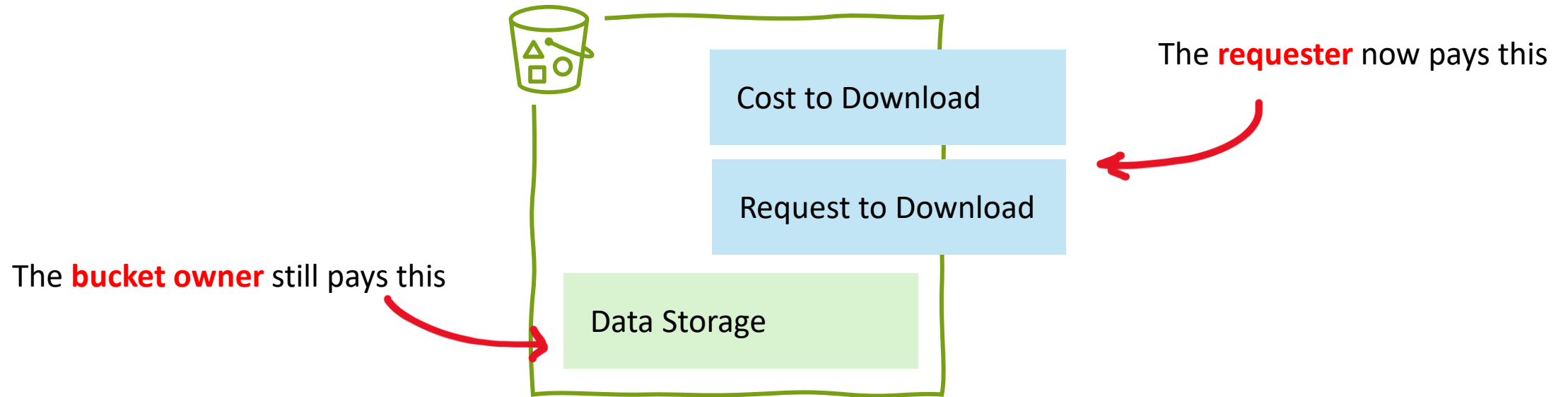
```
aws s3api restore-object \  
--bucket mybucket\  
--key myfile \  
--restore-request {}
```



Notice you **don't set** a type of retrieval or temporary amount of days

S3 – Requesters Pay

Requesters Pays bucket option allows the bucket owner to **offset specific S3 costs to the requester** (the user requesting the data).



When you want to **share data but not incur the charges associated with others accessing the data.** Eg.

- **Collaborative Projects:** External partners pay for their own S3 data uploads/downloads.
- **Client Data Storage:** Clients pay for their S3 storage and transfer costs.
- **Shared Educational Resources:** Researchers cover their S3 usage fees, not the institution.
- **Content Distribution:** Distributors/customers pay for S3 data transfer and downloads.

S3 – Requesters Pay

Requester pays

- ☒ **Disable**
Bucket owner pays the cost of data requests and data transfer.
- ☐ **Enable**
Requester will pay for requests and data transfer. access to this bucket is disabled.

You can at any time **toggle** Enable or Disable Requesters Pay on a bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::bucket-name/*",
      "Condition": {
        "StringEquals": {
          "s3:RequestPayer": "requester"
        }
      }
    }
  ]
}
```

- **All requests must authenticate** involving Requester Pays buckets
- Requester assumes an IAM role before making their request
 - The IAM policy will have a **s3:RequestPayer** condition.
- **anonymous access to that bucket is not allowed** on buckets with Requesters Pay
- The AWS account of the requester will be charged.

S3 – Requesters Pay Header

Requesters must include **x-amz-request-payer** in their API request header for:

- DELETE, GET, HEAD, POST, and PUT requests
- or as a parameter in a REST request

```
curl -X GET "https://bucket-name.s3.amazonaws.com/object-key" \  
      -H "x-amz-request-payer: requester" \  
      ...
```

In practice, you wouldn't use CURL for API requests since authentication is too challenging

In practice you will be setting the Requester Pay API requester header using the **AWS CLI and AWS SDK**

Using the AWS CLI, you can use the **--requester-payer** flag, so include the header in your object request

```
aws s3 cp \  
s3://bucket-name/object local/path/object \  
--request-payer requester
```

Setting the header using the AWS Ruby SDK

```
resp = s3_client.get_object(  
  bucket: bucket,  
  key: object_key,  
  request_payer: 'requester'  
)
```

S3 – Requesters Pay Troubleshooting



A **403 (Forbidden Request)** HTTP Error code will occur in the following scenarios:

- The requester doesn't include the parameter `x-amz-request-payer`
- Request authentication fails (something is wrong with the IAM role or IAM policy)
- The request is anonymous
- The request is a SOAP request. (SOAP requests are not allowed with requesters pay is turned on)



In the case the requesters forgets to include the header, then 403 will occur, and no charge will occur to the requester. No charge will occur to the bucket owner.

AWS Marketplace for S3



The AWS Marketplace for S3 provides alternatives to AWS Services that work with Amazon S3

AWS Services

Third-Party Services

Storage Backup and Recovery:

- FSx for Lustre
- Tape Gateway
- File Gateway

- Veeam Backup for AWS
- Druva: AWS Backup and Disaster Recovery

Data Integration and Analytics

- Transfer Family
- DataSync
- Athena

- ChoasSearch
- Logz.io
- BryteFlow Enterprise Edition

Observability and Monitoring

- CloudTrail
- CloudWatch

- DataDog, Splunk, Dynatrace

Security and Threat Detection

- GuardDuty
- Macie

- Trend Cloud One
- InsightIDR (Rapid7)
- VM-Series Virtual Next Generation Firewalls (NGFW) (Palo Alto Networks)

Permissions

- IAM

- OneLogin Workforce Identity
- FileCloud EFSS
- Yarkon S3 Server

S3 Batch Operations



S3 Batch Operations performs **large-scale** batch operations on Amazon S3 objects
billions of objects containing exabytes of data

The following **Batch Operation Types** can be performed:

- **Copy** Copies each object listed in the manifest to the specified destination bucket.
- **Invoke AWS Lambda function** Run a Lambda function against each object
- **Replace all object tags** Replaces the Amazon S3 object tags of each object
- **Replace access control list (ACL)** Replaces the (ACLs) for each object
- **Restore** Sends a restore request to S3 Glacier
- **Object Lock retention** Prevents overwriting or deleting for a fixed amount of time.
- **Object Lock legal hold** Prevents overwriting or deleting until the legal hold is removed.

Operation type

Choose the operation that you want to perform on all objects listed in the manifest. [Learn more](#)

- ☒ **Copy**
Copies each object listed in the manifest to the specified destination.
- ☐ **Invoke AWS Lambda function**
AWS Lambda is a compute service that lets you run code without provisioning or managing servers.
- ☐ **Replace all object tags**
Replaces the Amazon S3 object tags of each object listed in the manifest.
- ☐ **Replace access control list (ACL)**
Replaces the Amazon S3 access control lists (ACLs) for each object that is listed in the manifest.
- ☐ **Restore**
Sends a restore request to Amazon S3 Glacier for each object that is specified in the manifest.
- ☐ **Object Lock retention**
Prevents objects specified in the manifest from being deleted or overwritten for a fixed amount of time.
- ☐ **Object Lock legal hold**
Prevents objects specified in the manifest from being deleted or overwritten until the legal hold is removed.

Manifest format

- ☐ **S3 inventory report (manifest.json)**
- ☒ **CSV**
CSV format must be either 2 or 3 columns in

In order to perform a batch operation, you need to provide lists of objects in an S3 or supply an S3 Inventory report manifest.json.

You can have Batch Operation generate a **completion report** to audit the outcome of bulk operations.

Amazon S3 Inventory

Amazon S3 Inventory takes inventory of objects in an S3 bucket on a repeating schedule, so you have an audit history of object changes.

Amazon S3 will output the inventory into the destination of another S3 Bucket.

Frequency:

- Daily
 - delivered within 48 hours
- Weekly
 - First report delivered within 48 hours
 - Future reports every Sunday

Output format:

- **CSV** (Comma-separated values)
- **ORC** (Apache Optimized Columnar)
- **Parquet** (Apache Parquet)

Inventory Scope:

- Specific prefixes to filter objects
- Specific all or only current versions

You can **specify additional metadata** to be included in the report



Object	
☒	Size
☒	Last modified
☐	Multipart upload
☒	Replication status
☐	Encryption
☐	Bucket key status
Permissions	
☐	Object ACL
☐	Object owner
Storage class	
☐	Storage class
☐	Intelligent-Tiering: Access tier
Data integrity	
☐	ETag
☐	Additional checksums function
Object Lock	
☐	All Object Lock configurations
☐	Object Lock: Retention mode
☐	Object Lock: Retain until date
☐	Object Lock: Legal hold status

Amazon S3 Select



S3 Select lets you use a Structured Query Language **(SQL)** to filter the contents of S3 objects

works on objects stored in:

- CSV, JSON, or Apache Parquet format.

works with objects that are compressed with

- GZIP or BZIP2 (for CSV and JSON objects only)

works on object that are

- server-side encrypted objects

You can get results back in **JSON** or **CSV**

```
aws s3api select-object-content \  
--bucket my-bucket \  
--key my-data-file.csv \  
--expression "select * from s3object limit 100" \  
--expression-type 'SQL' \  
--input-serialization '{"CSV": {}, "CompressionType": "NONE"}' \  
--output-serialization '{"CSV": {}}' "output.csv"
```

It can be used in the following **Storage Classes**

- **S3 Standard**
- **S3 Intelligent-Tiering**
- **S3 Standard IA**
- **S3 One-Zone IA**
- ~~Reduced Redundancy Storage (RRS)~~
- **S3 Glacier Instant Retrieval**
- ~~S3 Glacier Flexible/Deep Retrieval~~
- ~~S3 Intelligent-Tiering Archive/Deep Archive Access~~

S3 Select vs Athena

S3 Select

Athena

Scope	Query contents of a single S3 Object	Query across multiple S3 objects and buckets.
Use case	Get a slice of data from a large file	Ad-hoc, analysis, BI tasks across large datasets
Query Complexity	Simple SQL expressions	Complex SQL eg. joins, window functions
Supported Formats	CSV, JSON, Parquet	CSV, JSON, ORC, Arvo, Parquet
Integrations	Used programmatically. No direct integrations with other AWS services	AWS Console UI, or programmatically, integrates with other AWS analytics services eg. AWS Glue.

S3 – Object Tags

S3 Object tags allow you to **apply tags at the object-level**
(unlike resource tags which are at the resource-level eg. buckets)

Key	Value
Project	ExamPro
Owner	Andrew
Year	2030
Environment	dev

Up to 10 Tags can be applied to an S3 object

Object Tags provide the following benefits:

- enable fine-grained access control of permissions in IAM policies
- enable fine-grained object lifecycle management in which you can specify a tag-based filter
- With S3 analytics, you can configure filters to group objects together for analysis by object tags,
- CloudWatch metrics can display information by specific tag filters

Amazon S3 Server Logs

Amazon S3 Server logs logging provides detailed records of the requests that are made to an Amazon S3 bucket.

Server access logging

☐ Disable

☒ Enable



S3 Server Logs can be **enabled on a bucket**.

There is no additional cost to enable S3 server logs except the S3 cost to store the logs in the destination bucket.

You'll need to set a destination bucket for the S3 Server logs.

Destination bucket:

- Can't have Payers request
- Can't have object locking

You can choose the **format of the logs**



Log object key format

- ☒ [DestinationPrefix][YYYY]-[MM]-[DD]-[hh]-[mm]-[ss]-[UniqueString]
- ☐ [DestinationPrefix][SourceAccountId]/[SourceRegion]/[SourceBucket]/[YYYY]/[MM]/[DD]/[YYYY]-[MM]-[DD]-[hh]-[mm]-[ss]-[UniqueString]

To speed up analytics and query applications, use this format.

Amazon S3 Server Logs

The log files are **plain text files** with each recorded object action on **each line** sorted by datetime

The log is very long...

79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be	my-bucket-ab	[06/Feb/2025:00:00:38 +0000]	192.0.2.3	79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be
79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be	my-bucket-ab	[06/Feb/2025:00:00:38 +0000]	192.0.2.3	79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be
79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be	my-bucket-ab	[06/Feb/2025:00:00:50 +0000]	192.0.2.3	79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be
79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be	my-bucket-ab	[06/Feb/2025:00:00:52 +0000]	192.0.2.3	79a59df900b949e55d96a1e698fbacedfd6e09d98eac8f8d5218e7cd47ef2be

- **Bucket Owner:** exampleb0f4e7a3e5b7395969a5
 - **Bucket Name:** my-awesome-bucket
 - **Time:** [06/Feb/2019:00:00:38 +0000]
 - **Remote IP:** 192.0.2.1
 - **Requester:** arn:aws:iam::123456789012:user/Alice
 - **Request ID:** 3E57427F4C1560EC
 - **Operation:** REST.GET.OBJECT
 - **Key:** photos/myphoto.jpg
 - **Request-URI:** GET /photos/myphoto.jpg HTTP/1.1
 - **HTTP Status:** 200
 - **Error Code:** NoSuchBucket
 - **Bytes Sent:** 1024
 - **Object Size:** 2048
 - **Total Time:** 100
 - **Turn-Around Time:** 50
 - **Referrer:** "https://www.example.com"
 - **User Agent:** "Mozilla/5.0 (compatible; Googlebot/2.1..
 - **Version ID:** 3HL4kqtJvjVBH40Nrjfd
 - **Host ID:** s9lzHYrFp76ZVxRcpX9+5cjAnEH2ROuNkd2B...
 - **Signature Version:** SigV4
 - **Cipher Suite:** ECDHE-RSA-AES128-GCM-SHA256
 - **Authentication Type:** AuthHeader
 - **Host Header:** s3.amazonaws.com
 - **TLS version:** TLSv1.2

There are additional fields for copy operations

CloudWatch metrics for S3

There are three types of CloudWatch metrics for S3

- 1. Storage metrics**

- free and on by default
- Reported once per day

- 2. Request metrics**

- extra costs
- needs to be enabled,
- Reported at one-minute interval
 - Latency processing so not realtime

- 3. Replication metrics**

- extra costs
- needs to be enabled

CloudWatch metrics for S3 – Requests metrics

- AllRequests
- GetRequests
- PutRequests
- DeleteRequests
- HeadRequests
- PostRequests
- SelectRequests
- SelectBytesScanned
- SelectBytesReturned
- ListRequests
- BytesDownloaded
- BytesUploaded
- 4xxErrors
- 5xxErrors
- FirstByteLatency
- TotalRequestLatency

You need to **manually enable** request metrics tracking.



```
aws s3api put-bucket-metrics-configuration \  
--endpoint https://s3.us-west-2.amazonaws.com \  
--bucket bucket-name \  
--id metrics-config-id \  
--metrics-configuration '{"Id":"metrics-config-id"}'
```

S3 Event Notifications

S3 Event Notifications allows your bucket to **notify** other AWS Services about s3 event data.

S3 Event Notifications makes **application integration very easy** for S3

Notification events

- New object created events
- Object removal events
- Restore object events
- Reduced Redundancy Storage (RRS) object lost events
- Replication events
- S3 Lifecycle expiration events
- S3 Lifecycle transition events
- S3 Intelligent-Tiering automatic archival events
- Object tagging events
- Object ACL PUT events

Possible Destination to other AWS Services

- Amazon Simple Notification Service (SNS) topics
- Amazon Simple Queue Service (SQS) queues
 - FIFO not supported
- AWS Lambda function
- Amazon EventBridge

Amazon S3 event notifications are **designed to be delivered at least once** notifications are delivered in seconds but can sometimes take a minute or longer

S3 Storage Class Analysis

Storage Class Analysis allows you to **analyze storage access patterns of objects** within a bucket to **recommend objects to move** between STANDARD to STANDARD_IA

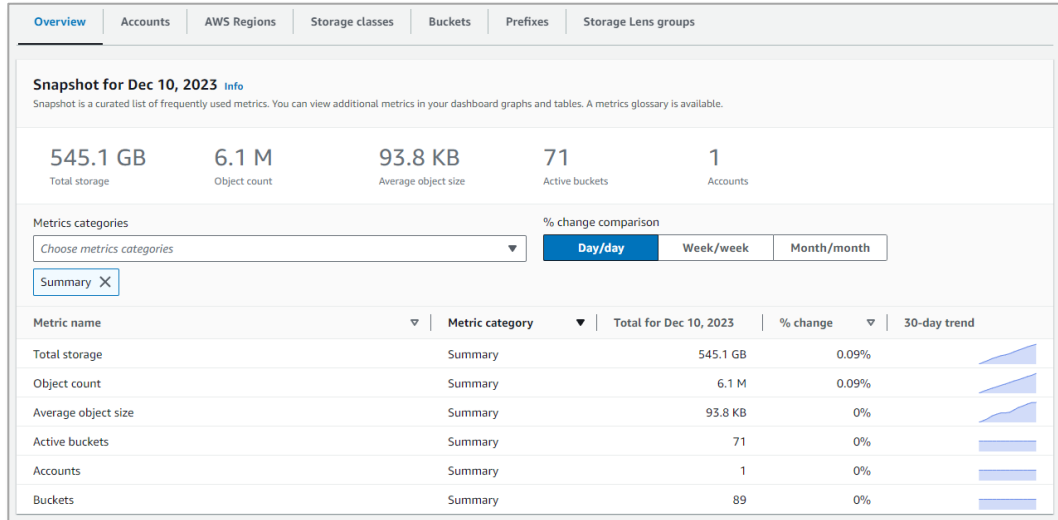
```
aws s3api put-bucket-analytics-configuration \  
--bucket my-bucket --id 1 \  
--analytics-configuration '{"Id": "1","StorageClassAnalysis": {}}'
```



- observes the infrequent access patterns of a filtered set of data over a period of time
- You can have multiple analysis filters per bucket (up to 1000 filters)
- The results can be exported to CSV
 - export this daily usage data to an S3 bucket
 - Use data in Amazon QuickSight for data visualization
- provides storage usage visualizations in the Amazon S3 console that are updated daily
- After a filter is applied, the analysis will be available 24 to 48 hours
- Storage Class analysis will analyze an object for 30 days or longer to gather enough information

S3 Storage Lens

Amazon S3 Storage Lens is a storage analysis tool for S3 buckets across your entire AWS organization.



- how much storage you have across your organization
- which are the fastest-growing buckets and prefixes
- identify cost-optimization opportunities
- implement data-protection and access-management best practices
- improve the performance of application workloads

- Metrics can be exported as CSV or Parquet to another S3 bucket
- Usage and metrics can be exported to Amazon CloudWatch

S3 Storage Lens aggregates metrics and displays the information in the **Account snapshot** as an **interactive dashboard** updated daily.

S3 Static Website Hosting

S3 Static Website Hosting allows you to **host and serve a static website from an S3 bucket.**

- S3 website endpoints do not support HTTPS
 - **Amazon CloudFront** must be used to serve HTTPS traffic



S3 Static Website hosting will provide a **website endpoint**

<http://bucket-name.s3-website-region.amazonaws.com>

<http://bucket-name.s3-website.region.amazonaws.com>

The format of the website endpoint varies based on region that will either have a hyphen or a period

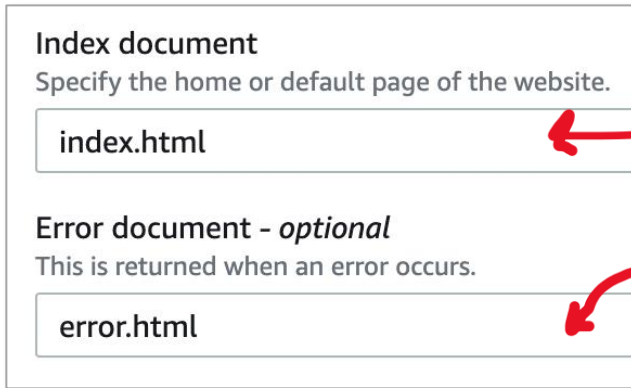
There are two hosting types (via the console):

- Host a static website
- Redirect requests to objects



Requester Pays buckets do not allow access through a website endpoint

S3 Static Website Hosting – Configure Hosting



Index document
Specify the home or default page of the website.

index.html

Error document - *optional*
This is returned when an error occurs.

error.html

By default, S3 Static Website hosting will serve **index.html** if found in the root of the bucket. You can override the index document.

You can also supply a custom error document **in the case of HTTP errors**. eg. 404 Not Found, 403 Forbidden

Redirection rules allow you to change how website content is served.

The rules can be written in either JSON or XML

- AWS Console only accepts JSON
- JSON or XML can be supplied via the AWS API
- Redirect based on status code
- Redirect based on prefix
- Redirect to a different protocol
- Redirect based on multiple conditions



```
{
  "RoutingRules": [
    {
      "Condition": {
        "KeyPrefixEquals": "product/old-product.html",
        "HttpErrorCodeReturnedEquals": "404"
      },
      "Redirect": {
        "HostName": "www.example.com",
        "ReplaceKeyWith": "product/new-product.html"
      }
    }
  ]
}
```



If you want a custom domain, you can do that through CloudFront

S3 Static Website Hosting – Redirect Requests

An S3 Bucket can be configured to **redirect all requests** to another S3 Bucket or personal domain.

Hosting type

- ☐ Host a static website
Use the bucket endpoint as the web address. [Learn more](#)
- ☒ Redirect requests for an object
Redirect requests to another bucket or domain. [Learn more](#)

Host name

exampro.co

Target bucket website address or personal domain

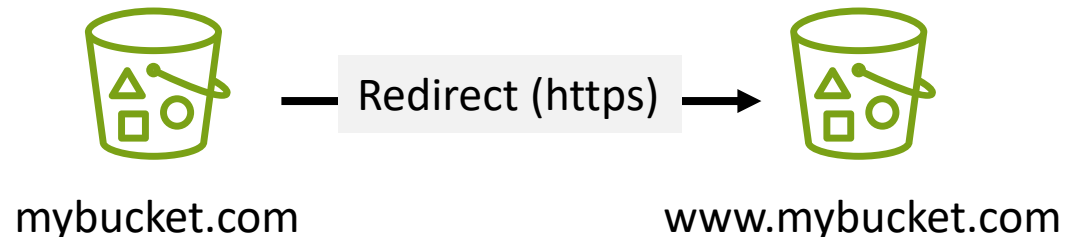
Protocol - *Optional*

- ☐ none
- ☐ http
- ☒ https



Not to be confused with Redirection rules for static website hosting which is for changing how per-object content is served.

Redirecting requests is a common strategy when setting up static website hosting since you will often create two buckets a **naked domain** and a **www.** domain with one pointing to another.



S3 Static Website Hosting – Public Access

An S3 bucket with Static Website Hosting **will not, by default, be publicly accessible.**

Without CloudFront

- Block Public Access needs to be turned off for Bucket Policies
- Bucket Policy read access for all of the internet

Only HTTP, no HTTPS

With CloudFront

- Block Public Access needs to be turned off for Bucket Policies
- Bucket Policy read access for all of the internet

- Website will be accessible via both S3 website endpoint and the CloudFront endpoint.
- Has HTTPS through CloudFront Endpoint

With CloudFront OAI (Origin Access Identity)

- Block Public Access needs to be turned off for Bucket Policies
- Bucket Policy read access for CloudFront Distribution.

- All traffic forced through CloudFront

With CloudFront OAC (Origin Access Controls)

- Block Public Access **does not** need to be turned on
- Bucket Policy read access to the OAC

Recommended

- All traffic forced through CloudFront
- More secure since Block Public Access can remain off.

S3 Static Website Hosting – Bucket Policy

In order for users to anonymously access your static website, a **bucket policy** will need to be configured.

Be able to **read files**

For **everything in the bucket.**

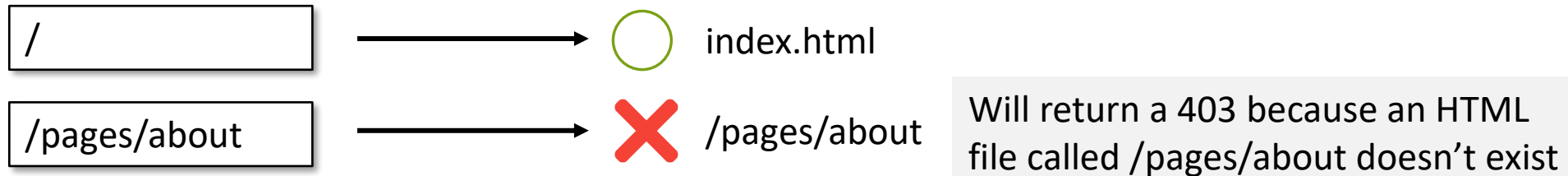
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::www.exampor.co/*"
      ]
    }
  ]
}
```

S3 Static Website Hosting – Single Page Apps

Single Page Applications (SPAs) can be deployed to S3 Static Website hosting.

What is a Single Page Application (SPA)?

When you have a single page eg. index.html, and we use JavaScript to dynamically create client-side other pages



In order for a SPA to work, you need to have CloudFront serving your S3 content

You need to set a **custom error page** in CloudFront so 403 will route to the document index page.

```
CustomErrorResponses:  
- ErrorCode: 403  
  ResponseCode: 200  
  ResponsePagePath: /index.html
```

Amazon S3 Byte Range Fetching

Amazon S3 allows you to **fetch a range of bytes** of data from S3 Objects using the Range header during S3 GetObject API Requests.

Example using the AWS SDK
Python (boto3) library



```
import boto3

s3 = boto3.client('s3')
bucket_name = 'your-bucket-name'
object_key = 'your-object-key'

# Get the first 100 bytes
byte_range = 'bytes=0-99'
response = s3.get_object(Bucket='mybucket', Key='myobject.txt', Range=byte_range)

# Read the partial content
data = response['Body'].read()
```

- Amazon S3 allows for concurrent connections so you can request multiple parts at the same time.
- Fetching smaller ranges of a large object also allows your application to improve retry times when requests are interrupted
- Typical sizes for byte-range requests are 8 MB or 16 MB

Amazon S3 Byte Range Fetching

You'll need to store each part and then concat all the parts downloaded in the correct order back into a single file.

Boto3 example opening multiple concurrent S3 connections, holding each part in memory and then reassembling the parts back into a single file.



Depending how large the file is you might need to write each part to disk if your program does not have enough memory to hold all parts.

```
import boto3

s3 = boto3.client('s3')
bucket = 'your-bucket-name'
key = 'your-object-key'

brange = ['bytes=0-99', 'bytes=100-199', 'bytes=200-299']

# Fetch the parts
parts = []
for byte_range in byte_ranges:
    response = s3.get_object(Bucket=bucket, Key=key, Range=brange)
    parts.append(response['Body'].read())

# Concatenate the parts
complete_file = b''.join(parts)

# Write the complete file to disk
with open('output_file', 'wb') as f:
    f.write(complete_file)
```

Amazon S3 Multipart Upload

Amazon S3 supports multipart upload, so you can **upload a single object in a set of parts.**

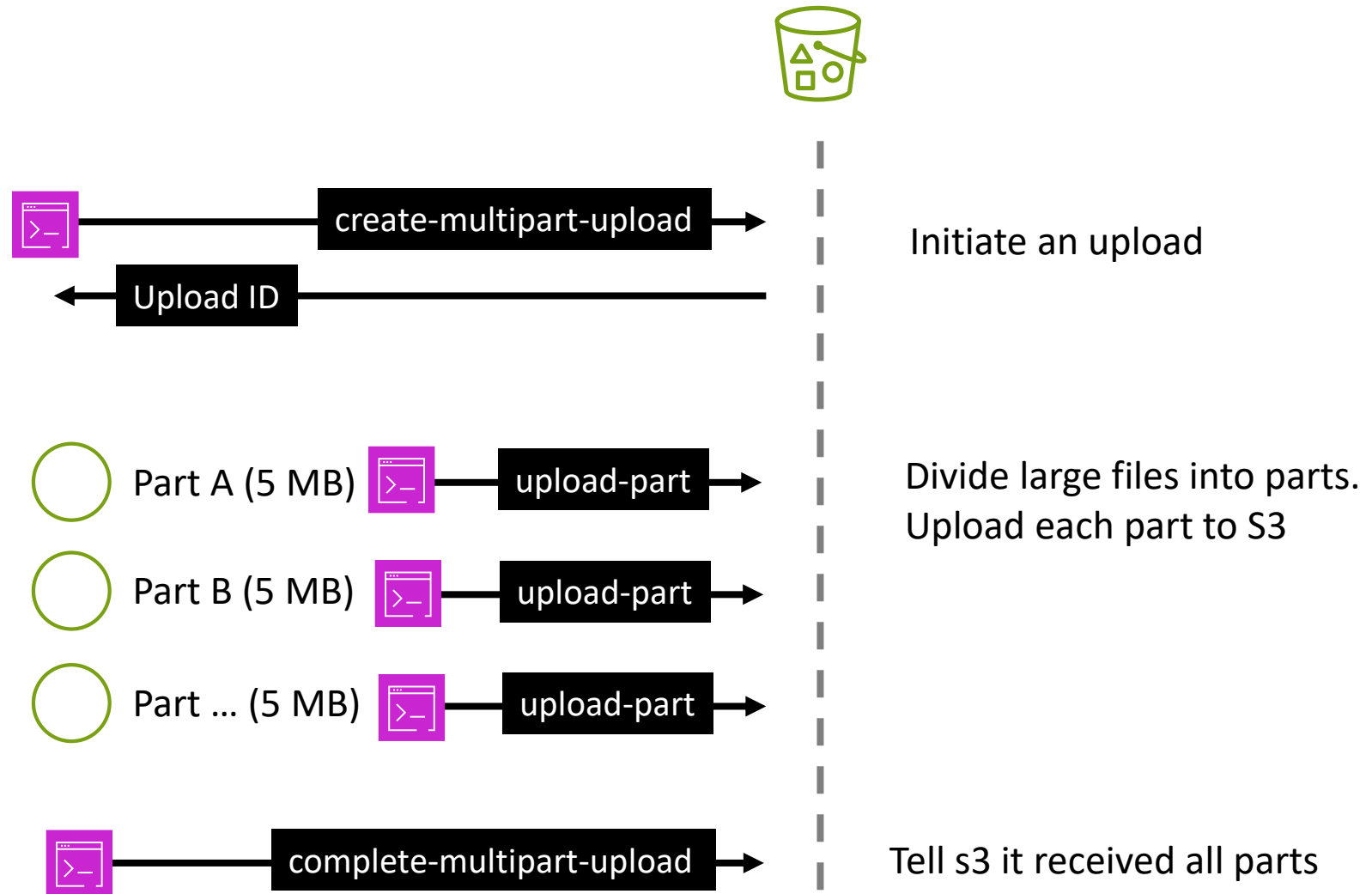
Multipart upload advantages:

- Improved throughput
- In case of network failure, you just need to reupload the missing parts
- Once you start a multipart upload, you can upload parts at any time, there is no expiry time to upload
- You can upload files while you're creating a file



For files that are +100MB, multipart upload is recommended

Amazon S3 Multipart Upload



Amazon S3 Multipart Upload

```
aws s3api create-multipart-upload \  
--bucket my-bucket \  
--key 'myfile'
```

We first need to initiate an upload, which will return back an **Upload ID**

```
aws s3api upload-part \  
--bucket my-bucket \  
--key 'myfile' \  
--part-number 1 \  
--body part01 \  
--upload-id "dfRtDYU0WWCCcH43C..."
```

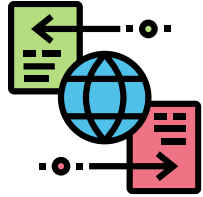
- We then upload each part by providing our Upload ID.
- Parts can be numbered from **1 to 1000**.
- We need to collect all the etags for each part.

```
aws s3api complete-multipart-upload \  
--bucket my-bucket \  
--key 'myfile' \  
--multipart-upload file://parts.json \  
--upload-id "dfRtDYU0WWCCcH43C..."
```

- Then we tell S3 we have finished upload
- We'll provide a **JSON file** with Etags corresponding to each part

```
{  
  "Parts": [  
    {  
      "PartNumber": 1, "ETag": "\"81e5e6e31c8430d7625c9c2b98e13c58\""  
    },  
    {  
      "PartNumber": 2, "ETag": "\"7f7c47e44d8c0f5f8c7d0c8b8e8b7f24\""  
    },  
    {  
      "PartNumber": 3, "ETag": "\"56ab24f6b5e0a50c2c3b9d58a3dabfb2\""  
    }  
  ]  
}
```

S3 Interoperability



What is Interoperability?

Interoperability in the context of cloud services is **the capability of cloud services to exchange and utilize information seamlessly with each other.**

Here are some common AWS services that often dump data into S3:

- **Amazon EC2:** Stores snapshots and backups in S3.
- **Amazon RDS:** Backups and data exports to S3.
- **AWS CloudTrail:** Stores API call logs in S3.
- **Amazon CloudWatch Logs:** Exports logs/metrics to S3.
- **AWS Lambda:** Outputs data/logs to S3.
- **AWS Glue:** ETL results stored in S3.
- **Amazon Kinesis:** Data streaming to S3 via Firehose.
- **Amazon EMR:** Uses S3 for input/output data storage.
- **Amazon Redshift:** Unloads data to S3.
- **AWS Data Pipeline:** Moves/transforms data to/from S3.
- **Amazon Athena:** Outputs query results to S3.
- **AWS IoT Core:** Stores IoT data in S3.